Mizar Evo

Readable, AI-Ready, Scalable Formal Mathematics

Eight problems, eight proposals

A discussion with the Bialystok Mizar team, September 2026

presenter notes edition

Mizar Evo project

September 2026

Mizar Evo

Mizar Evo
Readable, AI-Ready, Scalable Formal Mathematics
Eight problems, eight proposals
A discussion with the Bialystok Mizar team, September 2026
presenter notes edition

Mizar Evo project

September 2026

Presenter script:

- Thank the team.
- Explain that this is a design review. The team knows best whether the project still looks and reads like Mizar.
- The design is still open to change. We want to hear objections.

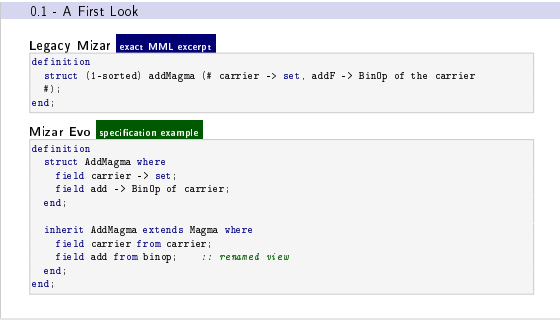
0.1 - A First Look

Legacy Mizar exact MML excerpt

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
  #);
end;
```

Mizar Evo specification example

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;     :: renamed view
  end;
end;
```



Presenter script: Say:

- Source: current MML `algstr_0.miz`, lines 37-40.
- This slide shows the main idea. It still reads as Mizar, and the mathematics is unchanged. The parent link and field mapping were implicit. Now they are visible in source text that the verifier can check.

0.2 - The Proposal In One Sentence

Key Phrase:

Preserve Mizar's mathematical vernacular.
Update the compiler, verifier, artifact, and publication layers.

What this talk does not claim:

- Full MML migration is not complete.
- The language standard is not final.
- AI assistance is not a substitute for proof checking.
- We build on what the current Mizar system has achieved.

Key Phrase:

Preserve Mizar's mathematical vernacular.
Update the compiler, verifier, artifact, and publication layers.

What this talk does not claim:

- Full MML migration is not complete.
- The language standard is not final.
- AI assistance is not a substitute for proof checking.
- We build on what the current Mizar system has achieved.

Presenter script: Read aloud:

- Full MML migration is not complete.
- The language standard is not final.
- AI assistance is not a substitute for proof checking.

After the example, explain which design obligation the syntax shows.

Every code example is labeled:

Label	Meaning
exact MML excerpt	exact current MML text, with article and line numbers
specification example	from the Mizar Evo language specification
sketch	an example; not yet fixed by the specification

Reading rule:

- No EBNF appears in this talk. The language is shown through examples only; `doc/spec/en/` remains the authority for grammar and edge cases.

deep dive

Presenter script: Read aloud:

- No EBNF appears in this talk. The language is shown through examples only; doc/spec/en/ remains the authority for grammar and edge cases.

For the table, read the row labels first, then the design reason.

0.3 - How To Read The Examples
deep dive

Every code example is labeled:

Label	Meaning
exact MML excerpt	exact current MML text, with article and line numbers
specification example	from the Mizar Evo language specification
sketch	an example; not yet fixed by the specification

Reading rule:

- No EBNF appears in this talk. The language is shown through examples only; doc/spec/en/ remains the authority for grammar and edge cases.

0.4 - What We Need From This Visit

Key Points:

- find compatibility constraints that are hard to see outside MML work;
- choose migration benchmark articles that are small but representative;
- review the trust boundary for ATP search and kernel checking;
- discuss how Formalized Mathematics should link to a package library;
- collect the objections we have not thought of.

Main question:

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

0.4 - What We Need From This Visit

Key Points:

- find compatibility constraints that are hard to see outside MML work;
- choose migration benchmark articles that are small but representative;
- review the trust boundary for ATP search and kernel checking;
- discuss how Formalized Mathematics should link to a package library;
- collect the objections we have not thought of.

Main question:

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

Presenter script: Read aloud:

- find compatibility constraints that are hard to see outside MML work;
- choose migration benchmark articles that are small but representative;
- review the trust boundary for ATP search and kernel checking;

After the example, explain which design obligation the syntax shows.

1.1 - What Mizar Got Right

Key Points:

- declarative proof text that reads as mathematics;
- soft types, modes, and adjective-rich vocabulary;
- attributes, registrations, and clusters as reusable automation;
- a well-established, carefully maintained library (MML 5.94.1493: 1493 articles);
- a tradition of publishing formal articles (Formalized Mathematics).

Message:

- We build on these strengths. We judge every proposal by whether it protects them.

1.1 - What Mizar Got Right

Key Points:
- declarative proof text that reads as mathematics;
- soft types, modes, and adjective-rich vocabulary;
- attributes, registrations, and clusters as reusable automation;
- a well-established, carefully maintained library (MML 5.94.1493: 1493 articles);
- a tradition of publishing formal articles (Formalized Mathematics).

Message:
- We build on these strengths. We judge every proposal by whether it protects them.

Presenter script: Say:

- The audience knows its own system. Spend one minute on the strengths we agree on.

Read aloud:

- declarative proof text that reads as mathematics;
- soft types, modes, and adjective-rich vocabulary;
- attributes, registrations, and clusters as reusable automation;

1.2 - Pressure One: Scale

Key Points:

- MML has grown to roughly 1500 articles that depend on each other.
- The unit of dependency, review, and reuse is the whole article.
- Tools resolve article environments. But reviewers cannot see the resolved dependencies in the source.
- Whole-library maintenance operations (renames, refactorings, revisions) become more risky as the library grows.

Message:

- The problem is not that current Mizar is wrong. It is that article-level boundaries were designed for a smaller library.

1.2 - Pressure One: Scale

Key Points:
- MML has grown to roughly 1500 articles that depend on each other.
- The unit of dependency, review, and reuse is the whole article.
- Tools resolve article environments. But reviewers cannot see the resolved dependencies in the source.
- Whole-library maintenance operations (renames, refactorings, revisions) become more risky as the library grows.

Message:
- The problem is not that current Mizar is wrong. It is that article-level boundaries were designed for a smaller library.

Presenter script: Read aloud:

- MML has grown to roughly 1500 articles that depend on each other.
- The unit of dependency, review, and reuse is the whole article.
- Tools resolve article environments. But reviewers cannot see the resolved dependencies in the source.
- Whole-library maintenance operations (renames, refactorings, revisions) become more risky as the library grows.
- The problem is not that current Mizar is wrong. It is that article-level boundaries were designed for a smaller library.

1.3 - Pressure Two: Tooling Expectations

Key Points:

- Editors are expected to give immediate feedback, even on incomplete or broken source.
- Builds are expected to be reproducible from a manifest and lockfile.
- Reuse is expected to work through packages with versions.
- Documentation is expected to be generated, linked, and browsable.

Message:

- Common programming tools already provide these features. New users expect formal libraries to provide them too.

Key Points:

- Editors are expected to give immediate feedback, even on incomplete or broken source.
- Builds are expected to be reproducible from a manifest and lockfile.
- Reuse is expected to work through packages with versions.
- Documentation is expected to be generated, linked, and browsable.

Message:

- Common programming tools already provide these features. New users expect formal libraries to provide them too.

Presenter script: Read aloud:

- Editors are expected to give immediate feedback, even on incomplete or broken source.
- Builds are expected to be reproducible from a manifest and lockfile.
- Reuse is expected to work through packages with versions.
- Documentation is expected to be generated, linked, and browsable.
- Common programming tools already provide these features. New users expect formal libraries to provide them too.

1.4 - Pressure Three: AI

Key Points:

- AI agents are already useful for search, explanation, and repair.
- They need a limited amount of structured context linked to the source, rather than a copy of the whole library.
- Their output must never decide whether a proof is valid. Verification must stay independent of how capable the assistant is.
- Readable source helps here. AI editing and retrieval work best with stable local text patterns.

Message:

- Mizar's readability makes safe AI assistance possible today.

1.4 - Pressure Three: AI

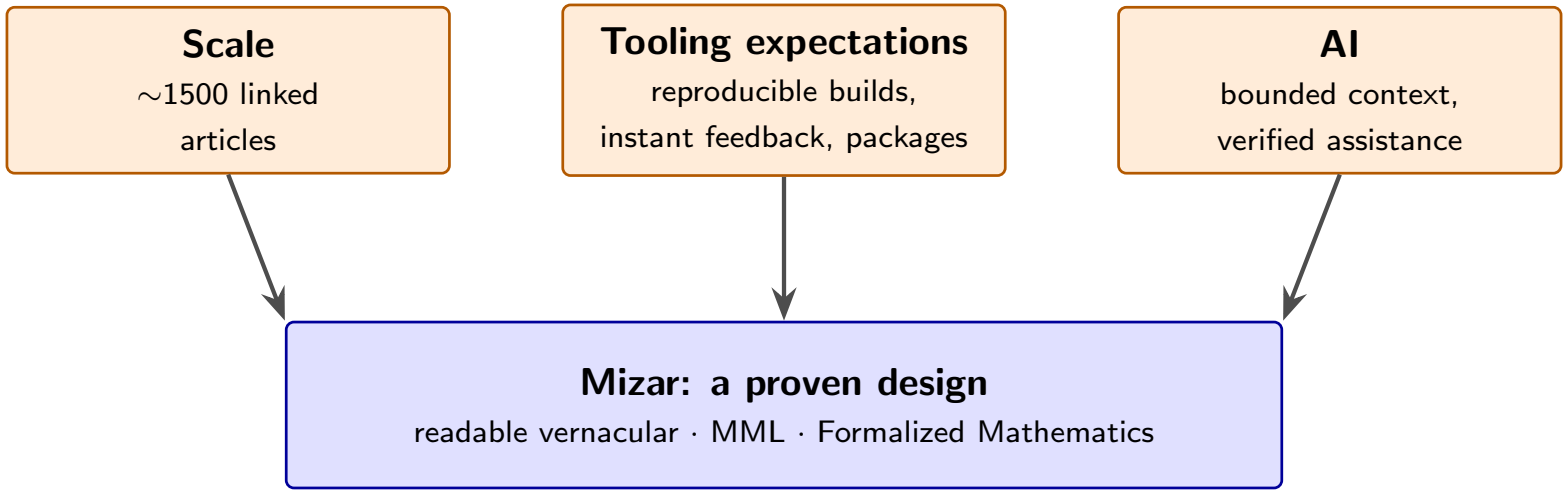
Key Points:
- AI agents are already useful for search, explanation, and repair.
- They need a limited amount of structured context linked to the source, rather than a copy of the whole library.
- Their output must never decide whether a proof is valid. Verification must stay independent of how capable the assistant is.
- Readable source helps here. AI editing and retrieval work best with stable local text patterns.

Message:
- Mizar's readability makes safe AI assistance possible today.

Presenter script: Read aloud:

- AI agents are already useful for search, explanation, and repair.
- They need a limited amount of structured context linked to the source, rather than a copy of the whole library.
- Their output must never decide whether a proof is valid. Verification must stay independent of how capable the assistant is.
- Readable source helps here. AI editing and retrieval work best with stable local text patterns.
- Mizar's readability makes safe AI assistance possible today.

1.5 - Three Pressures, One Design

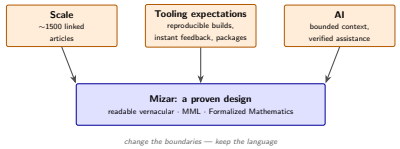


change the boundaries — keep the language

Message:

- These three needs do not mean that Mizar's design was wrong. Together, they give us reasons to change its boundaries.

1.5 - Three Pressures, One Design



Message:

- These three needs do not mean that Mizar's design was wrong. Together, they give us reasons to change its boundaries.

Presenter script: Read aloud:

- These three needs do not mean that Mizar's design was wrong. Together, they give us reasons to change its boundaries.

1.6 - The Design Rule

Key Phrase:

Do not make proofs harder to read to add automation.
Use automation to protect and extend readability.

Three goals, one test:

Goal	**Test for every feature**
Readability	does proof text still read as mathematics?
AI-readiness	can a tool see a limited context that we can audit?
Scalability	do boundaries stay stable as the library grows?

1.6 - The Design Rule

Key Phrase:

Do not make proofs harder to read to add automation.
Use automation to protect and extend readability.

Three goals, one test:

Goal	Test for every feature
Readability	does proof text still read as mathematics?
AI-readiness	can a tool see a limited context that we can audit?
Scalability	do boundaries stay stable as the library grows?

Presenter script: Say:

- The eight stories that follow each apply this rule to one specific problem.

For the table, read the row labels first, then the design reason.

Legacy Mizar exact MML excerpt

```
environ

 vocabularies XBOOLE_0, SUBSET_1, BINOP_1, ZFMISC_1, STRUCT_0, ARYTM_3,
      FUNCT_1, FUNCT_5, SUPINF_2, ARYTM_1, RELAT_1, MESFUNC1, ALGSTR_0, CARD_1;
 notations TARSKI, XBOOLE_0, SUBSET_1, ZFMISC_1, BINOP_1, FUNCT_5, ORDINAL1,
      CARD_1, STRUCT_0;
 constructors BINOP_1, STRUCT_0, ZFMISC_1, FUNCT_5;
 registrations ZFMISC_1, CARD_1, STRUCT_0;
 theorems STRUCT_0;

begin :: Additive structures
```

Legacy Mizar exact MML excerpt

```
environ

 vocabularies XBOOLE_0, SUBSET_1, BINOP_1, ZFMISC_1, STRUCT_0, ARYTM_3,
      FUNCT_1, FUNCT_5, SUPINF_2, ARYTM_1, RELAT_1, MESFUNC1, ALGSTR_0, CARD_1;
 notations TARSKI, XBOOLE_0, SUBSET_1, ZFMISC_1, BINOP_1, FUNCT_5, ORDINAL1,
      CARD_1, STRUCT_0;
 constructors BINOP_1, STRUCT_0, ZFMISC_1, FUNCT_5;
 registrations ZFMISC_1, CARD_1, STRUCT_0;
 theorems STRUCT_0;

begin :: Additive structures
```

Presenter script: Say:

- Source: current MML `algstr_0.miz`, lines 15-25.
- Everyone here has edited these blocks, trying changes until they work.
- `environ` has supported library growth for decades. We need to consider the cost of using it with today's larger library.

2.2 - Why This Is Difficult

deep dive

Key Points:

- Information about a symbol's origin is spread across several role lists. A reviewer cannot see which article provides which notation, constructor, or cluster.
- The Accommodator resolves the environment. But the resolved dependencies are not source text that a person can review.
- Tools cannot cache or invalidate units smaller than an article.
- Moving a theorem between articles risks breaking unknown dependents.

Message:

- Implicit dependencies add work to every edit, review, and tool. That cost grows with the library.

deep dive

2.2 - Why This Is Difficult deep dive

Key Points:
- Information about a symbol's origin is spread across several role lists. A reviewer cannot see which article provides which notation, constructor, or cluster.
- The Accommodator resolves the environment. But the resolved dependencies are not source text that a person can review.
- Tools cannot cache or invalidate units smaller than an article.
- Moving a theorem between articles risks breaking unknown dependents.

Message:
- Implicit dependencies add work to every edit, review, and tool. That cost grows with the library.

Presenter script: Read aloud:

- Information about a symbol's origin is spread across several role lists. A reviewer cannot see which article provides which notation, constructor, or cluster.
- The Accommodator resolves the environment. But the resolved dependencies are not source text that a person can review.
- Tools cannot cache or invalidate units smaller than an article.
- Moving a theorem between articles risks breaking unknown dependents.
- Implicit dependencies add work to every edit, review, and tool. That cost grows with the library.

2.3 - The Evo Answer: Import Prelude

Mizar Evo specification example

```
import .function;
import mml.algebra.structure.sorted;

definition
  let S be 1-sorted;
  mode BinOpDef: BinOp of S is
    Function of [: S.carrier, S.carrier :], S.carrier;
end;
```

Rules that make this deterministic:

- all imports appear before the first non-import item;
- imports provide the initial active lexicon. Local declarations extend it after their declaration points; imported items retain their source FQNs;
- stable fully-qualified names are derived from package and module paths.

2.3 - The Evo Answer: Import Prelude

Mizar Evo specification example

```
import .function;
import mml.algebra.structure.sorted;

definition
  let S be 1-sorted;
  mode BinOpDef: BinOp of S is
    Function of [: S.carrier, S.carrier :], S.carrier;
end;
```

Rules that make this deterministic:

- all imports appear before the first non-import item;
- imports provide the initial active lexicon. Local declarations extend it after their declaration points; imported items retain their source FQNs;
- stable fully-qualified names are derived from package and module paths.

Presenter script: Read aloud:

- all imports appear before the first non-import item;
- imports provide the initial active lexicon. Local declarations extend it after their declaration points; imported items retain their source FQNs;
- stable fully-qualified names are derived from package and module paths.

After the example, explain which design obligation the syntax shows.

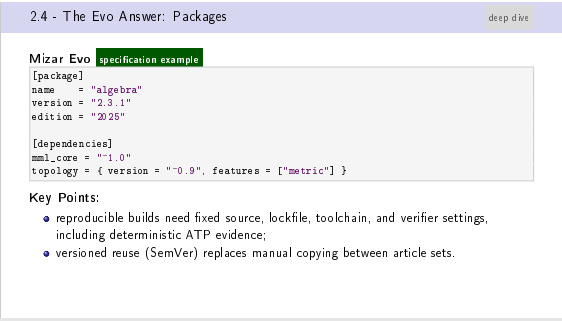
Mizar Evo specification example

```
[package]
name    = "algebra"
version = "2.3.1"
edition = "2025"

[dependencies]
mml_core = "^1.0"
topology = { version = "^0.9", features = ["metric"] }
```

Key Points:

- reproducible builds need fixed source, lockfile, toolchain, and verifier settings, including deterministic ATP evidence;
- versioned reuse (SemVer) replaces manual copying between article sets.

deep dive



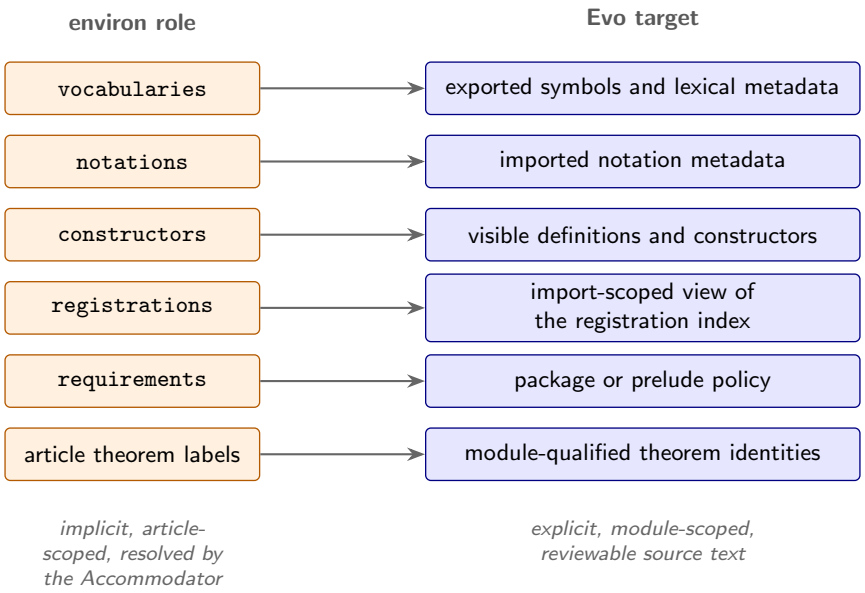
Presenter script: Read aloud:

- reproducible builds need fixed source, lockfile, toolchain, and verifier settings, including deterministic ATP evidence;
- versioned reuse (SemVer) replaces manual copying between article sets.

After the example, explain which design obligation the syntax shows.

2.5 - Migrating The Environment

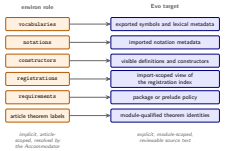
deep dive



Message:

- migration needs more than renaming. Current environments mix roles for semantics, syntax, and automation. Migration reports must explain what each imported module provides.

2.5 - Migrating The Environment
deep dive



Message:

- migration needs more than renaming. Current environments mix roles for semantics, syntax, and automation. Migration reports must explain what each imported module provides.

Presenter script: Read aloud:

- migration needs more than renaming. Current environments mix roles for semantics, syntax, and automation. Migration reports must explain what each imported module provides.

2.6 - What Is Preserved, What We Ask

Preserved:

- the mathematics and theorem identities are untouched;
- article-style authorship remains; a module is still a readable text;
- migration keeps origin metadata (story 8 returns to this).

Questions for Bialystok:

- Which `environ` roles must remain visually familiar during migration?
- Which current article dependencies are hardest to explain, and would make the best test cases for generated dependency reports?

2.6 - What Is Preserved, What We Ask

Preserved:

- the mathematics and theorem identities are untouched;
- article-style authorship remains; a module is still a readable text;
- migration keeps origin metadata (story 8 returns to this).

Questions for Bialystok:

- Which `environ` roles must remain visually familiar during migration?
- Which current article dependencies are hardest to explain, and would make the best test cases for generated dependency reports?

Presenter script: Read aloud:

- the mathematics and theorem identities are untouched;
- article-style authorship remains; a module is still a readable text;
- migration keeps origin metadata (story 8 returns to this).
- Which environ roles must remain visually familiar during migration?
- Which current article dependencies are hardest to explain, and would make the best test cases for generated dependency reports?

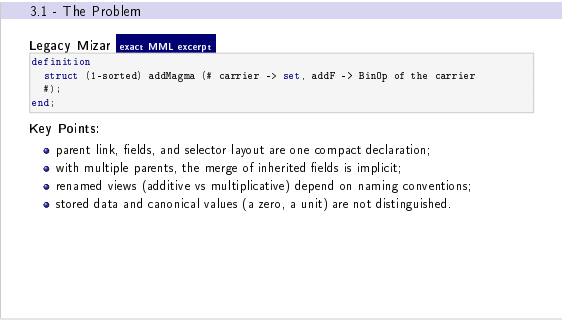
3.1 - The Problem

Legacy Mizar **exact MML excerpt**

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
  #);
end;
```

Key Points:

- parent link, fields, and selector layout are one compact declaration;
- with multiple parents, the merge of inherited fields is implicit;
- renamed views (additive vs multiplicative) depend on naming conventions;
- stored data and canonical values (a zero, a unit) are not distinguished.



Presenter script: Say:

- Source: current MML `algstr_0.miz`, lines 37-40.
- Explain why this short form was useful: structures stayed close to informal mathematical writing.

Read aloud:

- parent link, fields, and selector layout are one compact declaration;
- with multiple parents, the merge of inherited fields is implicit;
- renamed views (additive vs multiplicative) depend on naming conventions;

3.2 - Why This Is Difficult

deep dive

Key Points:

- Diamond inheritance (one structure reachable through two parent paths) is resolved by convention and declaration order, not by checkable source.
- The syntax does not tell a migration tool whether a selector is intrinsic data, a canonical value with obligations, or an inherited view.
- Errors appear far from their cause, as type mismatches in later articles.

Message:

- At MML scale, structure inheritance is a graph maintenance problem, and the graph needs explicit edges that the verifier can check.

deep dive

3.2 - Why This Is Difficult deep dive

Key Points:
- Diamond inheritance (one structure reachable through two parent paths) is resolved by convention and declaration order, not by checkable source.
- The syntax does not tell a migration tool whether a selector is intrinsic data, a canonical value with obligations, or an inherited view.
- Errors appear far from their cause, as type mismatches in later articles.

Message:
- At MML scale, structure inheritance is a graph maintenance problem, and the graph needs explicit edges that the verifier can check.

Presenter script: Read aloud:

- Diamond inheritance (one structure reachable through two parent paths) is resolved by convention and declaration order, not by checkable source.
- The syntax does not tell a migration tool whether a selector is intrinsic data, a canonical value with obligations, or an inherited view.
- Errors appear far from their cause, as type mismatches in later articles.
- At MML scale, structure inheritance is a graph maintenance problem, and the graph needs explicit edges that the verifier can check.

3.3 - The Evo Answer: Field, Property, Attribute

Mizar Evo specification example

```
definition
  struct AddLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    property zero -> Element of carrier;
  end;
end;
```

Three distinct concepts:

Concept	Meaning	Consequence
`field`	stored data	constructor arguments; equality of exact instances
`property`	derived value from an implementation	`means`: existence/uniqueness; `equals`: direct term
`attribute`	predicate-style refinement	cluster propagation, not layout

3.3 - The Evo Answer: Field, Property, Attribute

Mizar Evo specification example

```
definition
  struct AddLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    property zero -> Element of carrier;
  end;
end;
```

Three distinct concepts:

Concept	Meaning	Consequence
field	stored data	constructor arguments; equality of exact instances
property	derived value from an implementation	means: existence/uniqueness; equals: direct term
attribute	predicate-style refinement	cluster propagation, not layout

Presenter script: Say:

- A property declaration gives no value. Implementations supply values; constructors take fields only. Overlapping implementations need `coherence`.

For the table, read the row labels first, then the design reason.

3.4 - The Evo Answer: Explicit Inheritance

Mizar Evo specification example

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;     :: renamed
  end;
end;
```

- one `inherit` statement per parent; mapping and renaming are source text;
- non-identical inherited member types need `coherence` proofs of subtype inclusion. Same-name, same-type mappings need no proof;
- the additive/multiplicative naming convention becomes a checked view.

3.4 - The Evo Answer: Explicit Inheritance

Mizar Evo specification example

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;    :: renamed
  end;
end;
```


- one inherit statement per parent; mapping and renaming are source text;
- non-identical inherited member types need coherence proofs of subtype inclusion. Same-name, same-type mappings need no proof;
- the additive/multiplicative naming convention becomes a checked view.

Presenter script: Read aloud:

- one inherit statement per parent; mapping and renaming are source text;
- non-identical inherited member types need coherence proofs of subtype inclusion. Same-name, same-type mappings need no proof;
- the additive/multiplicative naming convention becomes a checked view.

After the example, explain which design obligation the syntax shows.

3.5 - The Evo Answer: Diamonds Become Checkable (1/2)

Mizar Evo specification example

```
definition
  struct DoubleLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    field mul -> BinOp of carrier;
    property zero -> Element of carrier;
    property one -> Element of carrier;
  end;

  inherit DoubleLoopStr extends AddLoopStr;
  inherit DoubleLoopStr extends MulLoopStr;
end;
```

Mizar Evo specification example

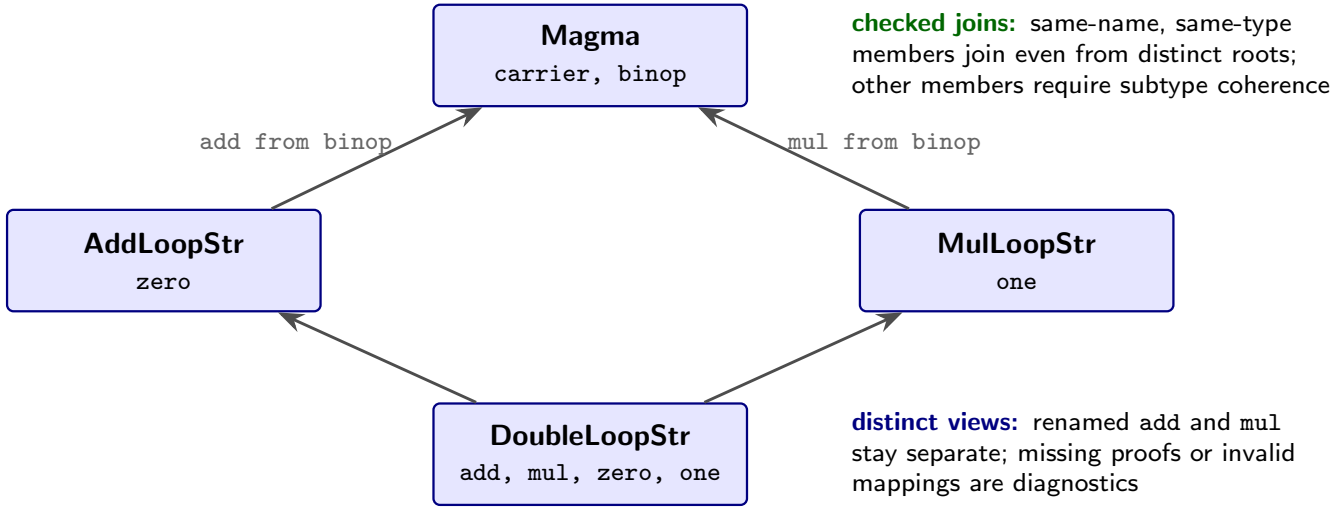
```
definition
  struct DoubleLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    field mul -> BinOp of carrier;
    property zero -> Element of carrier;
    property one -> Element of carrier;
  end;

  inherit DoubleLoopStr extends AddLoopStr;
  inherit DoubleLoopStr extends MulLoopStr;
end;
```

Presenter script: Say:

- The diagram assumes the shown parent mappings from `AddLoopStr` and `MulLoopStr` to `Magma`. The code adds their shared child.

3.5 - The Evo Answer: Diamonds Become Checkable (2/2)

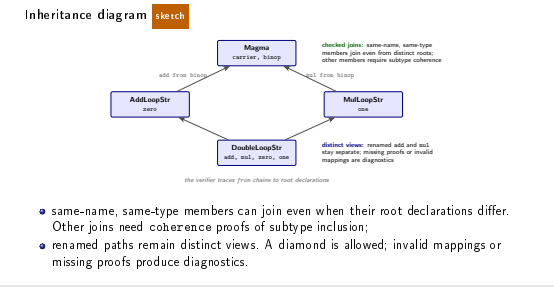
Inheritance diagram sketch



the verifier traces `from` chains to root declarations

- same-name, same-type members can join even when their root declarations differ. Other joins need `coherence` proofs of subtype inclusion;
- renamed paths remain distinct views. A diamond is allowed; invalid mappings or missing proofs produce diagnostics.

3.5 - The Evo Answer: Diamonds Become Checkable (2/2)



Presenter script: Read aloud:

- same-name, same-type members can join even when their root declarations differ. Other joins need coherence proofs of subtype inclusion;
- renamed paths remain distinct views. A diamond is allowed; invalid mappings or missing proofs produce diagnostics.

3.6 - What Is Preserved, What We Ask

Preserved:

- structures remain Mizar structures: carriers, selectors, `Element of`;
- aggregates become longer only where we need to make a hidden decision explicit.

Questions for Bialystok:

- Is the `field` / `property` / `attribute` split readable in real algebraic articles, or does it require too many annotations in simple cases?
- Is one-parent-per-`inherit` acceptable for parts of MML with much inheritance, such as the `ALGSTR` and topology hierarchies?
- Which MML structures would be the best diamond test cases?

3.6 - What Is Preserved, What We Ask

Preserved:

- structures remain Mizar structures: carriers, selectors, `Element of`;
- aggregates become longer only where we need to make a hidden decision explicit.

Questions for Bialystok:

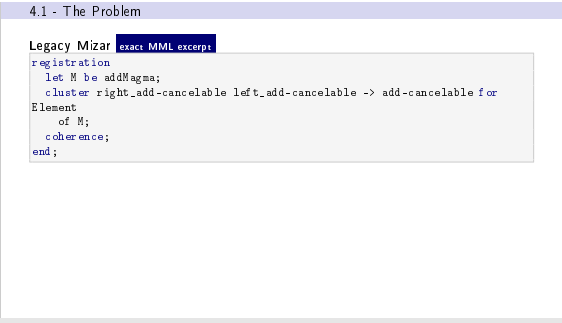
- Is the `field` / `property` / `attribute` split readable in real algebraic articles, or does it require too many annotations in simple cases?
- Is one-parent-per-`inherit` acceptable for parts of MML with much inheritance, such as the `ALGSTR` and topology hierarchies?
- Which MML structures would be the best diamond test cases?

Presenter script: Read aloud:

- structures remain Mizar structures: carriers, selectors, Element of;
- aggregates become longer only where we need to make a hidden decision explicit.
- Is the field / property / attribute split readable in real algebraic articles, or does it require too many annotations in simple cases?
- Is one-parent-per-inherit acceptable for parts of MML with much inheritance, such as the ALGSTR and topology hierarchies?
- Which MML structures would be the best diamond test cases?

Legacy Mizar exact MML excerpt

```
registration
  let M be addMagma;
  cluster right_add-cancelable left_add-cancelable -> add-cancelable for
Element
    of M;
  coherence;
end;
```



Presenter script: Say:

- Source: current MML `algstr_0.miz`, lines 104-109.
- Registrations are one of Mizar's best ideas. Adjectives propagate automatically, so proofs stay short. But it is hard to see how this works.

4.2 - Why This Is Difficult

deep dive

Key Points:

- When a proof fails, "why does the checker not see that this is a Group?" has no local answer. The cause is somewhere in the environment.
- Users cannot see which registrations fired or in which order. The automation is powerful, but explaining it becomes harder as it does more.
- For an AI assistant the situation is worse: it must guess the cluster state instead of reading it.

Message:

- Automation without explanations makes a large library harder to maintain, even when it is sound.

4.2 - Why This Is Difficult deep dive

Key Points:
- When a proof fails, "why does the checker not see that this is a Group?" has no local answer. The cause is somewhere in the environment.
- Users cannot see which registrations fired or in which order. The automation is powerful, but explaining it becomes harder as it does more.
- For an AI assistant the situation is worse: it must guess the cluster state instead of reading it.

Message:
- Automation without explanations makes a large library harder to maintain, even when it is sound.

Presenter script: Read aloud:

- When a proof fails, "why does the checker not see that this is a Group?" has no local answer. The cause is somewhere in the environment.
- Users cannot see which registrations fired or in which order. The automation is powerful, but explaining it becomes harder as it does more.
- For an AI assistant the situation is worse: it must guess the cluster state instead of reading it.
- Automation without explanations makes a large library harder to maintain, even when it is sound.

4.3 - The Evo Answer: Labeled, Traceable Registrations

Mizar Evo specification example

```
registration
  cluster EmptyImpliesFinite: empty -> finite for set;
  coherence proof ... end;

  cluster FiniteImpliesCountable: finite -> countable for set;
  coherence proof ... end;
end;
```

Key Points:

- every registration item carries a required label: citable in by, reported in diagnostics, part of the module interface;
- the verifier uses an import-filtered view of the global cluster graph;
- explain-attribute and resolution traces show why an attribute follows or why resolution fails, e.g. empty -> finite -> countable.

4.3 - The Evo Answer: Labeled, Traceable Registrations

Mizar Evo specification example

```
registration
  cluster EmptyImpliesFinite: empty -> finite for set;
  coherence proof ... end;

  cluster FiniteImpliesCountable: finite -> countable for set;
  coherence proof ... end;
end;
```

Key Points:

- every registration item carries a required label: citable in by, reported in diagnostics, part of the module interface;
- the verifier uses an import-filtered view of the global cluster graph;
- explain-attribute and resolution traces show why an attribute follows or why resolution fails, e.g. empty -> finite -> countable.

Presenter script: Read aloud:

- every registration item carries a required label: citable in by, reported in diagnostics, part of the module interface;
- the verifier uses an import-filtered view of the global cluster graph;
- explain-attribute and resolution traces show why an attribute follows or why resolution fails, e.g. empty -> finite -> countable.

After the example, explain which design obligation the syntax shows.

Mizar Evo specification example

```
registration
  let n be Nat;
  reduce NatAddZero: n + 0 to n;
  reducibility
  proof
    let n be Nat;
    thus n + 0 = n by mml.number.natural.Nat_add_zero;
  end;
end;
```

Key Points:

- a reduction is an oriented simplification backed by an equality proof;
- the right side must be strictly smaller, so imported rules cannot loop, and rule selection is deterministic (pattern subsumption, then guard specificity, then FQN tie-break);
- unoriented identification idioms become auditable `reduce` items.

deep dive

4.4 - The Evo Answer: Oriented Reductions deep dive

Mizar Evo specification example

```
registration
  let n be Nat;
  reduce NatAddZero: n + 0 to n;
  reducibility
  proof
    let n be Nat;
    thus n + 0 = n by mml.number.natural.Nat_add_zero;
  end;
end;
```

Key Points:

- a reduction is an oriented simplification backed by an equality proof;
- the right side must be strictly smaller, so imported rules cannot loop, and rule selection is deterministic (pattern subsumption, then guard specificity, then FQN tie-break);
- unoriented identification idioms become auditable reduce items.

Presenter script: Read aloud:

- a reduction is an oriented simplification backed by an equality proof;
- the right side must be strictly smaller, so imported rules cannot loop, and rule selection is deterministic (pattern subsumption, then guard specificity, then FQN tie-break);
- unoriented identification idioms become auditable reduce items.

After the example, explain which design obligation the syntax shows.

4.5 - What Is Preserved, What We Ask

Preserved:

- registrations and clusters remain first-class; proofs stay short;
- cluster applications need no repeated proofs. Reductions may need local guard evidence or an explicit equality citation.

Questions for Bialystok:

- Which cluster explanations would help most with daily work: failure explanations, firing traces, or difference reports between environments?
- Which MML article families make the most complex use of registrations and should become migration benchmarks for the cluster graph?

4.5 - What Is Preserved, What We Ask

Preserved:

- registrations and clusters remain first-class; proofs stay short;
- cluster applications need no repeated proofs. Reductions may need local guard evidence or an explicit equality citation.

Questions for Bialystok:

- Which cluster explanations would help most with daily work: failure explanations, firing traces, or difference reports between environments?
- Which MML article families make the most complex use of registrations and should become migration benchmarks for the cluster graph?

Presenter script: Read aloud:

- registrations and clusters remain first-class; proofs stay short;
- cluster applications need no repeated proofs. Reductions may need local guard evidence or an explicit equality citation.
- Which cluster explanations would help most with daily work: failure explanations, firing traces, or difference reports between environments?
- Which MML article families make the most complex use of registrations and should become migration benchmarks for the cluster graph?

5.1 - The Problem

Key Points:

- Users want stronger automation: bigger by steps, hammer-style search.
- But in a monolithic verifier, every gain in search power grows the code that must be trusted.
- External provers (ATPs) are strong exactly where Mizar's core is first-order - and they are the least auditable component of all.
- MizAR and MPTP research already showed that ATP search is powerful on MML premises; the open question is trust, not power.

Key Phrase:

How do we get modern proof search
without trusting the searcher?

5.1 - The Problem

Key Points:
- Users want stronger automation: bigger by steps, hammer-style search.
- But in a monolithic verifier, every gain in search power grows the code that must be trusted.
- External provers (ATPs) are strong exactly where Mizar's core is first-order - and they are the least auditable component of all.
- MizAR and MPTP research already showed that ATP search is powerful on MML premises; the open question is trust, not power.

Key Phrase:

How do we get modern proof search without trusting the searcher?

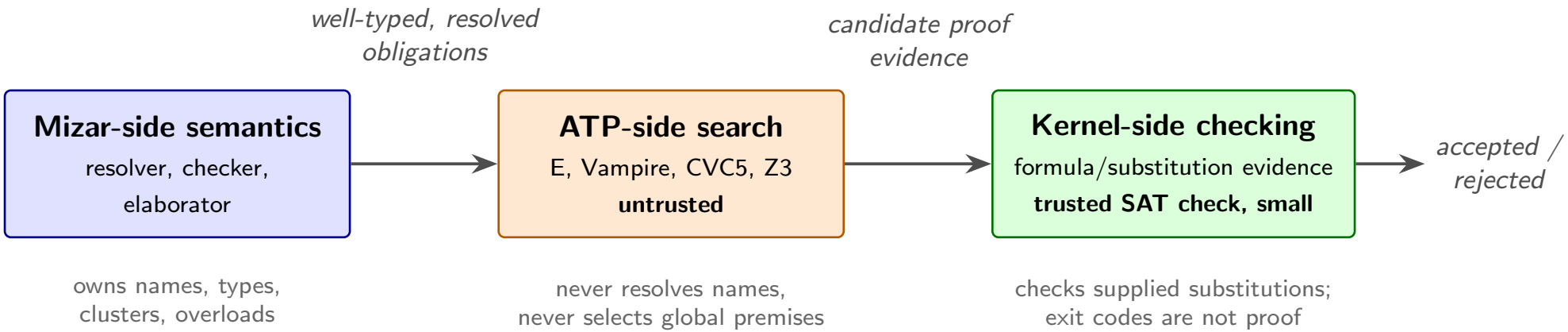
Presenter script: Say:

- Mention MizAR, MPTP, and hammer research by name. They showed how well search works on MML. Mizar Evo adds a boundary that allows us to use that search without making the trusted base larger.

Read aloud:

- Users want stronger automation: bigger by steps, hammer-style search.
- But in a monolithic verifier, every gain in search power grows the code that must be trusted.
- External provers (ATPs) are strong exactly where Mizar's core is first-order - and they are the least auditable component of all.

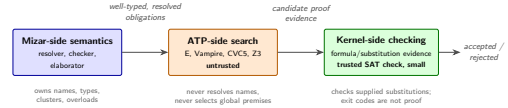
5.2 - The Evo Answer: A Reasoning Boundary



Key rules:

- ATPs never resolve names, infer types, expand clusters, or pick overloads;
- the kernel checks supplied formulas and substitutions, then runs its trusted SAT check. It does not select premises or invent substitutions;
- deterministic pre-ATP discharge needs replayable evidence too - nothing is accepted because an earlier phase said "done".

5.2 - The Evo Answer: A Reasoning Boundary



Key rules:

- ATPs never resolve names, infer types, expand clusters, or pick overloads;
- the kernel checks supplied formulas and substitutions, then runs its trusted SAT check. It does not select premises or invent substitutions;
- deterministic pre-ATP discharge needs replayable evidence too - nothing is accepted because an earlier phase said "done".

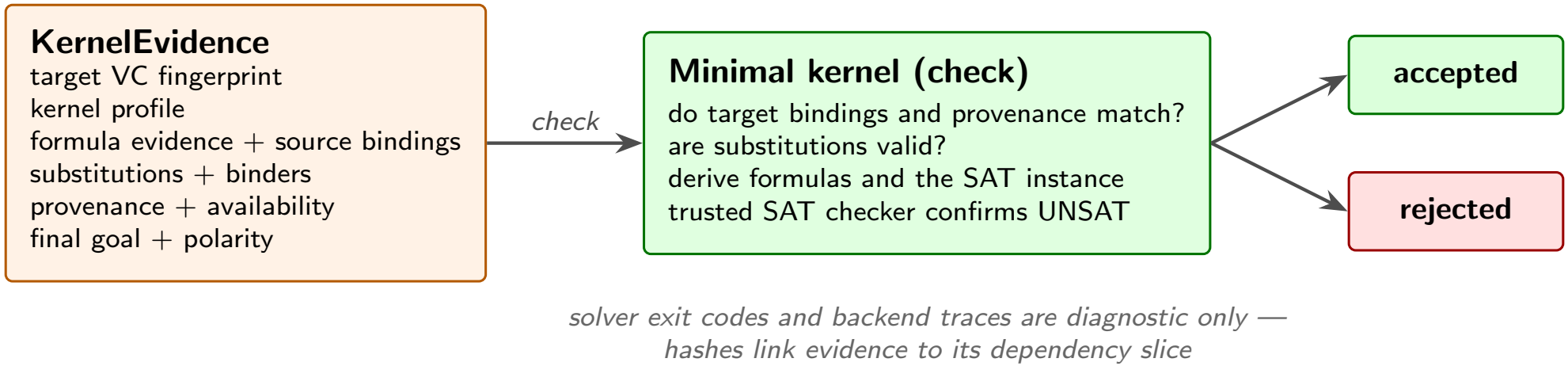
Presenter script: Say:

- Development policy may record `externally_attested` results. These are separate from kernel-verified proofs.

Read aloud:

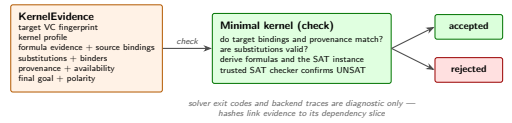
- ATPs never resolve names, infer types, expand clusters, or pick overloads;
- the kernel checks supplied formulas and substitutions, then runs its trusted SAT check. It does not select premises or invent substitutions;
- deterministic pre-ATP discharge needs replayable evidence too - nothing is accepted because an earlier phase said "done".

5.3 - Formula And Substitution Evidence



- `KernelEvidence` contains source formulas, explicit substitutions, provenance, and target/goal bindings;
- the kernel checks this evidence, derives instantiated formulas and SAT clauses, and requires UNSAT from its trusted in-process Rust SAT checker;
- backend resolution traces, SMT proof objects, logs, and exit codes are not trusted acceptance evidence.

5.3 - Formula And Substitution Evidence



- `KernelEvidence` contains source formulas, explicit substitutions, provenance, and target/goal bindings;
- the kernel checks this evidence, derives instantiated formulas and SAT clauses, and requires UNSAT from its trusted in-process Rust SAT checker;
- backend resolution traces, SMT proof objects, logs, and exit codes are not trusted acceptance evidence.

Presenter script: Say:

- Hashes bind evidence to its dependency context. The kernel also checks binder conditions and the goal's refutation polarity.

Read aloud:

- KernelEvidence contains source formulas, explicit substitutions, provenance, and target/goal bindings;
- the kernel checks this evidence, derives instantiated formulas and SAT clauses, and requires UNSAT from its trusted in-process Rust SAT checker;
- backend resolution traces, SMT proof objects, logs, and exit codes are not trusted acceptance evidence.

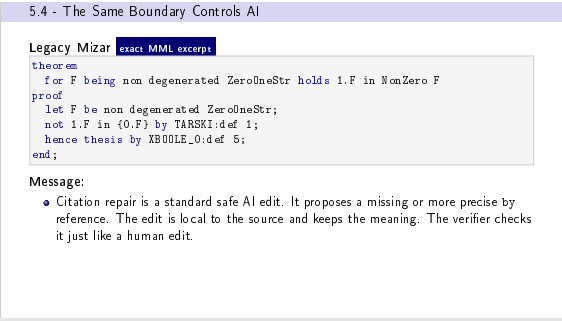
5.4 - The Same Boundary Controls AI

Legacy Mizar **exact MML excerpt**

```
theorem
  for F being non degenerated ZeroOneStr holds 1.F in NonZero F
proof
  let F be non degenerated ZeroOneStr;
  not 1.F in {0.F} by TARSKI:def 1;
  hence thesis by XBOOLE_0:def 5;
end;
```

Message:

- Citation repair is a standard safe AI edit. It proposes a missing or more precise by reference. The edit is local to the source and keeps the meaning. The verifier checks it just like a human edit.



Presenter script: Say:

- Source: current MML `struct_0.miz`, lines 637-643.
- The agent proposes; the verifier and kernel decide. The assistant's strength never enters the trusted base.

Read aloud:

- Citation repair is a standard safe AI edit. It proposes a missing or more precise by reference. The edit is local to the source and keeps the meaning. The verifier checks it just like a human edit.

5.5 - Edit Classes: Green, Yellow, Red

deep dive

Class	Examples	Policy
Green	add citation, insert qua, info annotation	can be proposed automatically; still verified
Yellow	add import, local lemma, registration	proposed with human review
Red	weaken theorem, change definition, add axiom	forbidden to ordinary agents

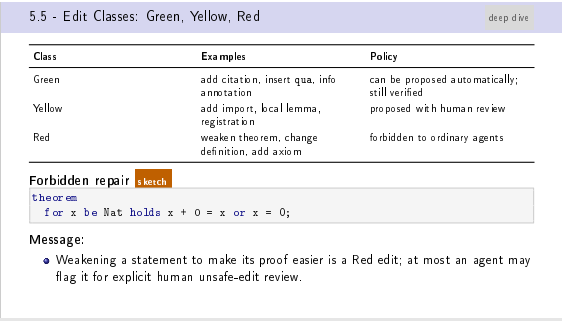
Forbidden repair sketch

```
theorem
  for x be Nat holds x + 0 = x or x = 0;
```

Message:

- Weakening a statement to make its proof easier is a Red edit; at most an agent may flag it for explicit human unsafe-edit review.

deep dive



Presenter script: Read aloud:

- Weakening a statement to make its proof easier is a Red edit; at most an agent may flag it for explicit human unsafe-edit review.

After the example, explain which design obligation the syntax shows.
For the table, read the row labels first, then the design reason.

5.6 - What Is Preserved, What We Ask

Preserved:

- the de Bruijn discipline: acceptance uses a small trusted core, including its SAT checker;
- proof text stays declarative and readable - automation maintains the argument, it does not replace it.

Questions for Bialystok:

- Is the formula/substitution evidence approach convincing for Mizar-style obligations, including clusters and definitional expansions?
- Which evidence format would the team be most willing to audit?

5.6 - What Is Preserved, What We Ask

Preserved:
- the de Bruijn discipline: acceptance uses a small trusted core, including its SAT checker;
- proof text stays declarative and readable - automation maintains the argument, it does not replace it.

Questions for Bialystok:
- Is the formula/substitution evidence approach convincing for Mizar-style obligations, including clusters and definitional expansions?
- Which evidence format would the team be most willing to audit?

Presenter script: Read aloud:

- the de Bruijn discipline: acceptance uses a small trusted core, including its SAT checker;
- proof text stays declarative and readable - automation maintains the argument, it does not replace it.
- Is the formula/substitution evidence approach convincing for Mizar-style obligations, including clusters and definitional expansions?
- Which evidence format would the team be most willing to audit?

6.1 - The Problem

Key Points:

- Verifying the whole MML is a batch operation measured in hours.
- The reuse boundary is the accepted article: a small change re-verifies more than it should.
- Memory follows the article environment, not the actually used interface.
- These costs follow from using the article as the unit in a large library. They are not errors in the current design.

6.1 - The Problem

Key Points:
- Verifying the whole MML is a batch operation measured in hours.
- The reuse boundary is the accepted article: a small change re-verifies more than it should.
- Memory follows the article environment, not the actually used interface.
- These costs follow from using the article as the unit in a large library. They are not errors in the current design.

Presenter script: Read aloud:

- Verifying the whole MML is a batch operation measured in hours.
- The reuse boundary is the accepted article: a small change re-verifies more than it should.
- Memory follows the article environment, not the actually used interface.
- These costs follow from using the article as the unit in a large library. They are not errors in the current design.

6.2 - The Evo Answer: Fingerprints And Incrementality (1/2)

Key Points:

- cache keys cover source, dependency slices, package/lockfile, toolchain, schema, policy, registrations, obligations, evidence, and witnesses;
- reuse requires all relevant keys to match; missing data causes a cache miss;
- a theorem proof-body edit does not rebuild importers if its exported statement and accepted status stay unchanged;
- independent modules, obligations, ATP runs, and kernel checks run in parallel; results are published in canonical order.

6.2 - The Evo Answer: Fingerprints And Incrementality (1/2)

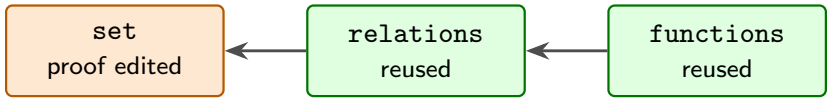
Key Points:
- cache keys cover source, dependency slices, package/lockfile, toolchain, schema, policy, registrations, obligations, evidence, and witnesses;
- reuse requires all relevant keys to match; missing data causes a cache miss;
- a theorem proof-body edit does not rebuild importers if its exported statement and accepted status stay unchanged;
- independent modules, obligations, ATP runs, and kernel checks run in parallel; results are published in canonical order.

Presenter script: Read aloud:

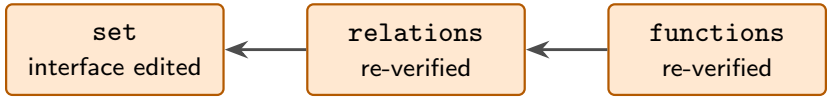
- cache keys cover source, dependency slices, package/lockfile, toolchain, schema, policy, registrations, obligations, evidence, and witnesses;
- reuse requires all relevant keys to match; missing data causes a cache miss;
- a theorem proof-body edit does not rebuild importers if its exported statement and accepted status stay unchanged;
- independent modules, obligations, ATP runs, and kernel checks run in parallel; results are published in canonical order.

6.2 - The Evo Answer: Fingerprints And Incrementality (2/2)

theorem proof edit:
public statement + accepted status unchanged — importers reuse

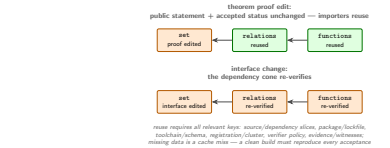


interface change:
the dependency cone re-verifies



reuse requires all relevant keys: source/dependency slices, package/lockfile, toolchain/schema, registration/cluster, verifier policy, evidence/witnesses; missing data is a cache miss — a clean build must reproduce every acceptance

Cache reuse is never proof authority.
A clean build must reproduce every acceptance.



Cache reuse is never proof authority.
A clean build must reproduce every acceptance.

Presenter script: After the example, explain which design obligation the syntax shows. Introduce the next topic: The Evo Answer: Fingerprints And Incrementality (2/2). Point to the example, question, or diagram before you continue.

6.3 - The Evo Answer: A Memory Contract

deep dive

```
resident memory should scale with:
  active source and typed AST
  imported public interfaces
  import-filtered indexes
  active module obligations
  in-flight proof checks and ATP runs
  small caches

not with:
  imported proof bodies
  imported proof witnesses
  private lemmas outside the interface
  registration data outside the import closure
```

Message:

- This is a resident-memory model, not a measured performance guarantee. Proof bodies and traces load lazily for specific queries.

deep dive

6.3 - The Evo Answer: A Memory Contract
deep dive

```
resident memory should scale with:
  active source and typed AST
  imported public interfaces
  import-filtered indexes
  active module obligations
  in-flight proof checks and ATP runs
  small caches

not with:
  imported proof bodies
  imported proof witnesses
  private lemmas outside the interface
  registration data outside the import closure
```

Message:

- This is a resident-memory model, not a measured performance guarantee. Proof bodies and traces load lazily for specific queries.

Presenter script: Read aloud:

- This is a resident-memory model, not a measured performance guarantee. Proof bodies and traces load lazily for specific queries.

After the example, explain which design obligation the syntax shows.

6.4 - What Is Preserved, What We Ask

Preserved:

- clean-build semantics: caching and parallelism change speed, never truth;
- article-style review: what a human reads is still complete source text.

Questions for Bialystok:

- What clean-build equivalence tests would make incremental verification trustworthy to the team that maintains MML today?
- Which current maintenance operations (revisions, renamings) should we benchmark for incremental cost?

6.4 - What Is Preserved, What We Ask

Preserved:
- clean-build semantics: caching and parallelism change speed, never truth;
- article-style review: what a human reads is still complete source text.

Questions for Bialystok:
- What clean-build equivalence tests would make incremental verification trustworthy to the team that maintains MML today?
- Which current maintenance operations (revisions, renamings) should we benchmark for incremental cost?

Presenter script: Read aloud:

- clean-build semantics: caching and parallelism change speed, never truth;
- article-style review: what a human reads is still complete source text.
- What clean-build equivalence tests would make incremental verification trustworthy to the team that maintains MML today?
- Which current maintenance operations (revisions, renamings) should we benchmark for incremental cost?

7.1 - The Problem, Part One: Schemes Use Separate Rules

Legacy Mizar **exact MML excerpt**

```
scheme
  NatInd { P[Nat] } : for k being Nat holds P[k]
provided
A1: P[0] and
A2: for k be Nat st P[k] holds P[k + 1]
```

Key Points:

- schemes support second-order patterns (induction, separation, replacement). They work, but they use a separate mechanism with separate rules;
- schemes can parameterize theorems, but not structures, modes, or functors.

7.1 - The Problem, Part One: Schemes Use Separate Rules

Legacy Mizar exact MML excerpt

```
scheme
  NatInd { P[Nat] } : for k being Nat holds P[k]
provided
A1: P[0] and
A2: for k be Nat st P[k] holds P[k + 1]
```

Key Points:

- schemes support second-order patterns (induction, separation, replacement). They work, but they use a separate mechanism with separate rules;
- schemes can parameterize theorems, but not structures, modes, or functors.

Presenter script: Say:

- Source: current MML `nat_1.miz`, near line 90 (checked July 2, 2026).

Read aloud:

- schemes support second-order patterns (induction, separation, replacement). They work, but they use a separate mechanism with separate rules;
- schemes can parameterize theorems, but not structures, modes, or functors.

7.2 - The Problem, Part Two: Copy-Paste Algebra

Key Points:

- `addMagma` and `multMagma` are the same mathematics twice, related only by naming convention (we saw this in story 2);
- polynomial rings, vector spaces, and matrix theories are written again for each carrier because there is no parameterized construction;
- a theorem proved for one commutative operation is re-proved for + and $*$ separately.

Message:

- Without a generics mechanism, the library needs duplicate articles. Each copy needs maintenance.

7.2 - The Problem, Part Two: Copy-Paste Algebra

Key Points:
- addMagma and multMagma are the same mathematics twice, related only by naming convention (we saw this in story 2);
- polynomial rings, vector spaces, and matrix theories are written again for each carrier because there is no parameterized construction;
- a theorem proved for one commutative operation is re-proved for + and * separately.

Message:
- Without a generics mechanism, the library needs duplicate articles. Each copy needs maintenance.

Presenter script: Read aloud:

- addMagma and multMagma are the same mathematics twice, related only by naming convention (we saw this in story 2);
- polynomial rings, vector spaces, and matrix theories are written again for each carrier because there is no parameterized construction;
- a theorem proved for one commutative operation is re-proved for + and * separately.
- Without a generics mechanism, the library needs duplicate articles. Each copy needs maintenance.

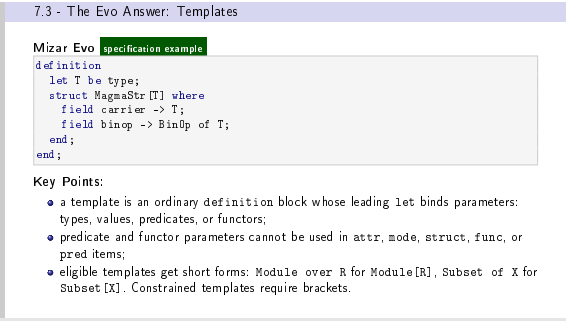
7.3 - The Evo Answer: Templates

Mizar Evo specification example

```
definition
  let T be type;
  struct MagmaStr[T] where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;
```

Key Points:

- a template is an ordinary `definition` block whose leading `let` binds parameters: types, values, predicates, or functors;
- predicate and functor parameters cannot be used in `attr`, `mode`, `struct`, `func`, or `pred` items;
- eligible templates get short forms: `Module over R` for `Module[R]`, `Subset of X` for `Subset[X]`. Constrained templates require brackets.



Presenter script: Read aloud:

- a template is an ordinary definition block whose leading let binds parameters: types, values, predicates, or functors;
- predicate and functor parameters cannot be used in attr, mode, struct, func, or pred items;
- eligible templates get short forms: Module over R for Module[R], Subset of X for Subset[X]. Constrained templates require brackets.

After the example, explain which design obligation the syntax shows.

7.4 - Bounded Parameters And Generic Theorems

deep dive

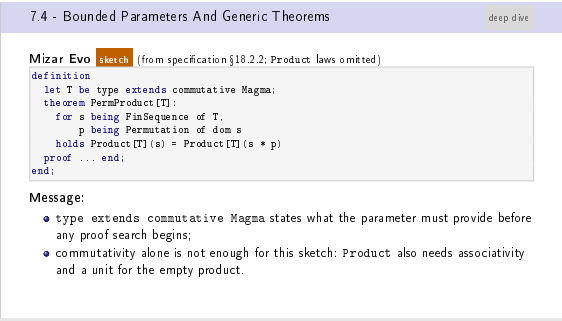
Mizar Evo sketch (from specification §18.2.2; `Product` laws omitted)

```
definition
  let T be type extends commutative Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
        p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

Message:

- `type extends commutative Magma` states what the parameter must provide before any proof search begins;
- commutativity alone is not enough for this sketch: `Product` also needs associativity and a unit for the empty product.

deep dive



Presenter script: Read aloud:

- Mizar Evo (sketch from specification §18.2.2; Product laws omitted):
- type extends commutative Magma states what the parameter must provide before any proof search begins;
- commutativity alone is not enough for this sketch: Product also needs associativity and a unit for the empty product.

After the example, explain which design obligation the syntax shows.

7.5 - One Proof, Many Instantiations

deep dive

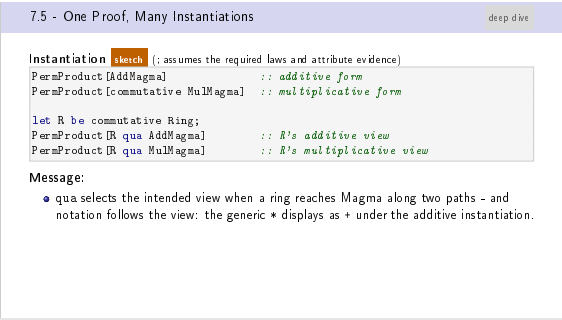
Instantiation sketch (; assumes the required laws and attribute evidence)

```
PermProduct[AddMagma]                 :: additive form
PermProduct[commutative MulMagma]     :: multiplicative form

let R be commutative Ring;
PermProduct[R qua AddMagma]           :: R's additive view
PermProduct[R qua MulMagma]           :: R's multiplicative view
```

Message:

- qua selects the intended view when a ring reaches Magma along two paths - and notation follows the view: the generic $*$ displays as + under the additive instantiation.

deep dive



Presenter script: Read aloud:

- Instantiation (sketch; assumes the required laws and attribute evidence):
- qua selects the intended view when a ring reaches Magma along two paths - and notation follows the view: the generic * displays as + under the additive instantiation.

After the example, explain which design obligation the syntax shows.

Mizar Evo specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

Key Points:

- predicate parameters follow the familiar `defpred` convention;
- legacy schemes have a direct, mechanical migration target;
- theorem instantiation uses explicit brackets, so tools and artifacts see exactly which instance a proof uses.

deep dive

7.6 - Schemes Become Ordinary Templates deep dive

Mizar Evo specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

Key Points:

- predicate parameters follow the familiar defpred convention;
- legacy schemes have a direct, mechanical migration target;
- theorem instantiation uses explicit brackets, so tools and artifacts see exactly which instance a proof uses.

Presenter script: Read aloud:

- predicate parameters follow the familiar defpred convention;
- legacy schemes have a direct, mechanical migration target;
- theorem instantiation uses explicit brackets, so tools and artifacts see exactly which instance a proof uses.

After the example, explain which design obligation the syntax shows.

7.7 - What Is Preserved, What We Ask

Preserved:

- scheme-style reasoning keeps the same power;
- `of` / `over` phrasing keeps mathematical prose readable;
- first-order discipline: each instantiation is checked; templates add no new logic.

Questions for Bialystok:

- Which MML schemes should be the first migration targets?
- Are brackets acceptable as the canonical identity form, with `of`/`over` as display forms?
- For `func` and `pred` templates, normalized declared argument types must determine each inferred type parameter uniquely. `qua` views are never inferred. Does this rule require too many explicit `[T]` arguments?

7.7 - What Is Preserved, What We Ask

Preserved:

- scheme-style reasoning keeps the same power;
- of / over phrasing keeps mathematical prose readable;
- first-order discipline: each instantiation is checked; templates add no new logic.

Questions for Bialystok:

- Which MML schemes should be the first migration targets?
- Are brackets acceptable as the canonical identity form, with of/over as display forms?
- For `func` and `pred` templates, normalized declared argument types must determine each inferred type parameter uniquely. `qua` views are never inferred. Does this rule require too many explicit `[T]` arguments?

Presenter script: Read aloud:

- scheme-style reasoning keeps the same power;
- of / over phrasing keeps mathematical prose readable;
- first-order discipline: each instantiation is checked; templates add no new logic.
- Which MML schemes should be the first migration targets?
- Are brackets acceptable as the canonical identity form, with of/over as display forms?

8.1 - The Problem

Key Points:

- Current Mizar can define `Gcd` and prove its theory - but cannot compute `gcd(48, 18)`; every numeric fact needs a hand-written proof chain.
- There is no checked connection between MML mathematics and executable code; verified-algorithm work must leave the system entirely.
- This is a boundary of the design, not a defect: Mizar chose to be a proof language. The question is whether that boundary still meets our needs.

Key Phrase:

The library describes computation.
It cannot perform or export it.

8.1 - The Problem

Key Points:

- Current Mizar can define Gcd and prove its theory - but cannot compute gcd(48, 18); every numeric fact needs a hand-written proof chain.
- There is no checked connection between MML mathematics and executable code; verified-algorithm work must leave the system entirely.
- This is a boundary of the design, not a defect: Mizar chose to be a proof language. The question is whether that boundary still meets our needs.

Key Phrase:

The library describes computation.
It cannot perform or export it.

Presenter script: Read aloud:

- Current Mizar can define Gcd and prove its theory - but cannot compute gcd(48, 18); every numeric fact needs a hand-written proof chain.
- There is no checked connection between MML mathematics and executable code; verified-algorithm work must leave the system entirely.
- This is a boundary of the design, not a defect: Mizar chose to be a proof language. The question is whether that boundary still meets our needs.

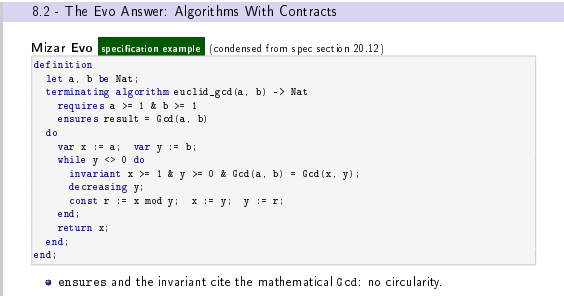
After the example, explain which design obligation the syntax shows.

8.2 - The Evo Answer: Algorithms With Contracts

Mizar Evo **specification example** (condensed from spec section 20.12)

```
definition
  let a, b be Nat;
  terminating algorithm euclid_gcd(a, b) -> Nat
    requires a >= 1 & b >= 1
    ensures result = Gcd(a, b)
  do
    var x := a;  var y := b;
    while y <> 0 do
      invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y);
      decreasing y;
      const r := x mod y;  x := y;  y := r;
    end;
    return x;
  end;
end;
```

- ensures and the invariant cite the mathematical Gcd: no circularity.



Presenter script: Say:

- The contract uses mathematics. The algorithm computes the result, and the library functor `Gcd` specifies it. The `decreasing` measure on `y` proves termination.

Read aloud:

- ensures and the invariant cite the mathematical Gcd: no circularity.

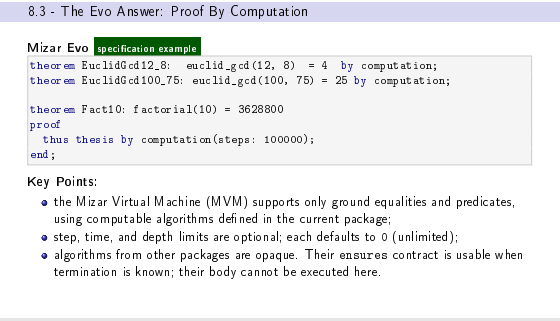
8.3 - The Evo Answer: Proof By Computation

Mizar Evo specification example

```
theorem EuclidGcd12_8:  euclid_gcd(12, 8)  = 4  by computation;
theorem EuclidGcd100_75: euclid_gcd(100, 75) = 25 by computation;

theorem Fact10: factorial(10) = 3628800
proof
  thus thesis by computation(steps: 100000);
end;
```

Key Points:

- the Mizar Virtual Machine (MVM) supports only ground equalities and predicates, using computable algorithms defined in the current package;
- step, time, and depth limits are optional; each defaults to 0 (unlimited);
- algorithms from other packages are opaque. Their `ensures` contract is usable when termination is known; their body cannot be executed here.



Presenter script: Read aloud:

- the Mizar Virtual Machine (MVM) supports only ground equalities and predicates, using computable algorithms defined in the current package;
- step, time, and depth limits are optional; each defaults to 0 (unlimited);
- algorithms from other packages are opaque. Their ensures contract is usable when termination is known; their body cannot be executed here.

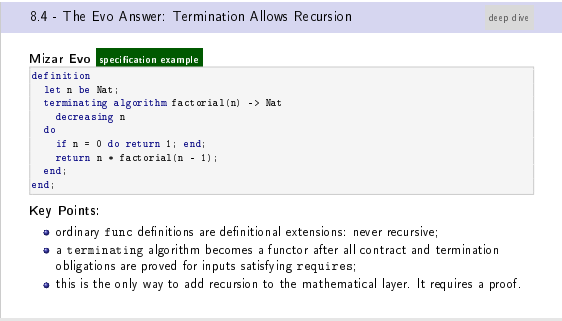
After the example, explain which design obligation the syntax shows.

Mizar Evo specification example

```
definition
  let n be Nat;
  terminating algorithm factorial(n) -> Nat
    decreasing n
  do
    if n = 0 do return 1; end;
    return n * factorial(n - 1);
  end;
end;
```

Key Points:

- ordinary `func` definitions are definitional extensions: never recursive;
- a `terminating` algorithm becomes a functor after all contract and termination obligations are proved for inputs satisfying `requires`;
- this is the only way to add recursion to the mathematical layer. It requires a proof.



Presenter script: Say:

- Each call must satisfy `requires`. An ordinary algorithm may prove only partial correctness and is not promoted to a functor.

Read aloud:

- ordinary func definitions are definitional extensions: never recursive;
- a terminating algorithm becomes a functor after all contract and termination obligations are proved for inputs satisfying requires;
- this is the only way to add recursion to the mathematical layer. It requires a proof.

8.5 - Computation Never Redefines Truth

deep dive

Key Points:

- contracts, invariants, and termination measures generate verification conditions that pass through the same ATP-plus-kernel boundary as ordinary theorems (story 4);
- `by computation` uses MVM replay with optional limits;
- code extraction (to runtime targets) is strictly downstream of verified artifacts and can never affect acceptance.

Message:

- Algorithms let the library express more. They do not change what it means for a theorem to be accepted.

└8.5 - Computation Never Redefines Truth

deep dive

Presenter script: Read aloud:

- contracts, invariants, and termination measures generate verification conditions that pass through the same ATP-plus-kernel boundary as ordinary theorems (story 4);
- by computation uses MVM replay with optional limits;
- code extraction (to runtime targets) is strictly downstream of verified artifacts and can never affect acceptance.
- Algorithms let the library express more. They do not change what it means for a theorem to be accepted.

8.5 - Computation Never Redefines Truth
deep dive
Key Points:
- contracts, invariants, and termination measures generate verification conditions that pass through the same ATP-plus-kernel boundary as ordinary theorems (story 4);
- by computation uses MVM replay with optional limits;
- code extraction (to runtime targets) is strictly downstream of verified artifacts and can never affect acceptance.
Message:
- Algorithms let the library express more. They do not change what it means for a theorem to be accepted.

8.6 - What Is Preserved, What We Ask

Preserved:

- algorithms live in `definition` blocks. Proofs can use verified promotion or `by computation` for computable ground calls in the current package;
- a partial algorithm's `ensures` needs evidence that the call terminates;
- first-order set-theoretic foundations stay intact.

Questions for Bialystok:

- Which computational examples would show clear benefits to mathematicians without shifting the culture toward programming?
- Are there MML areas (number theory, combinatorics, finite structures) where `by computation` would immediately shorten real proofs?
- Which extraction targets matter first, if any?

8.6 - What Is Preserved, What We Ask

Preserved:

- algorithms live in `definition` blocks. Proofs can use verified promotion or by `computation` for computable ground calls in the current package;
- a partial algorithm's ensures needs evidence that the call terminates;
- first-order set-theoretic foundations stay intact.

Questions for Bialystok:

- Which computational examples would show clear benefits to mathematicians without shifting the culture toward programming?
- Are there MML areas (number theory, combinatorics, finite structures) where by `computation` would immediately shorten real proofs?
- Which extraction targets matter first, if any?

Presenter script: Read aloud:

- algorithms live in definition blocks. Proofs can use verified promotion or by computation for computable ground calls in the current package;
- a partial algorithm's ensures needs evidence that the call terminates;
- first-order set-theoretic foundations stay intact.
- Which computational examples would show clear benefits to mathematicians without shifting the culture toward programming?
- Are there MML areas (number theory, combinatorics, finite structures) where by computation would immediately shorten real proofs?

9.1 - The Problem

Key Points:

- Formalized Mathematics gives Mizar a rare feature: a research journal linked to a formal library, with articles that people can cite.
- But article identity is closely tied to library organization. Refactoring the library can conflict with the structure of published articles. Package reuse has no identity for journal citations.
- A written explanation needs an order that helps readers. Reuse needs dependency order. One structure cannot serve both as well as possible.

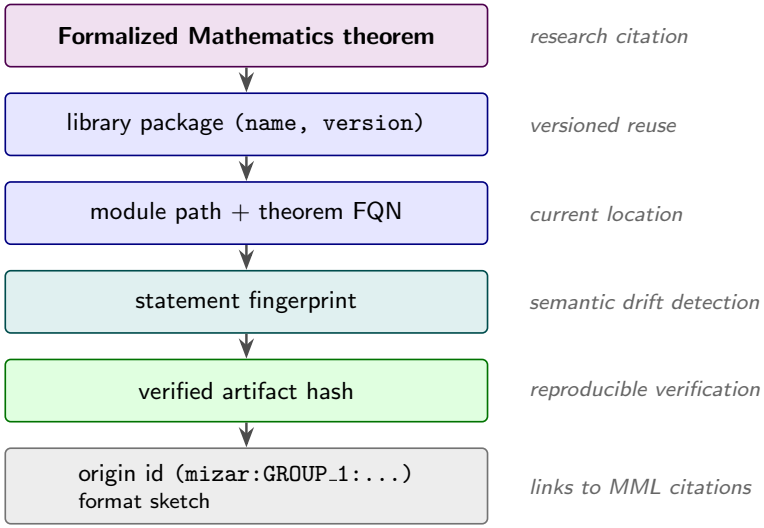
9.1 - The Problem

Key Points:
- Formalized Mathematics gives Mizar a rare feature: a research journal linked to a formal library, with articles that people can cite.
- But article identity is closely tied to library organization. Refactoring the library can conflict with the structure of published articles. Package reuse has no identity for journal citations.
- A written explanation needs an order that helps readers. Reuse needs dependency order. One structure cannot serve both as well as possible.

Presenter script: Read aloud:

- Formalized Mathematics gives Mizar a rare feature: a research journal linked to a formal library, with articles that people can cite.
- But article identity is closely tied to library organization. Refactoring the library can conflict with the structure of published articles. Package reuse has no identity for journal citations.
- A written explanation needs an order that helps readers. Reuse needs dependency order. One structure cannot serve both as well as possible.

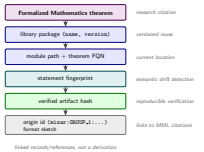
9.2 - The Evo Proposal: Linked Records



Message:

- each layer answers a different question: research citation, current location, semantic drift detection, reproducible verification, and links to MML and past Formalized Mathematics volumes.

9.2 - The Evo Proposal: Linked Records



Message:

- each layer answers a different question: research citation, current location, semantic drift detection, reproducible verification, and links to MML and past Formalized Mathematics volumes.

Presenter script: Read aloud:

- each layer answers a different question: research citation, current location, semantic drift detection, reproducible verification, and links to MML and past Formalized Mathematics volumes.

9.3 - Who Gains What

deep dive

Audience	**Gain**
readers	written explanations come first; formal source is one click away
maintainers	refactoring no longer rewrites published explanations
authors	frozen `pub` article identities; FQNs for library locations
planned AI tools	prose for retrieval, fingerprints for exact context

9.3 - Who Gains What deep dive

Audience	Gain
readers	written explanations come first; formal source is one click away
maintainers	refactoring no longer rewrites published explanations
authors	frozen pub article identities; FQNs for library locations
planned AI tools	prose for retrieval, fingerprints for exact context

Presenter script: For the table, read the row labels first, then the design reason. Introduce the next topic: Who Gains What. Point to the example, question, or diagram before you continue.

9.4 - What Is Preserved, What We Ask

Preserved:

- Formalized Mathematics remains a real journal with review and written explanations;
- proposed origin metadata links migrated items to MML citations; the migration mapping still needs to be defined.

Questions for Bialystok:

- Which identity should be primary in user-facing citations: article label, library FQN, or origin id?
- How should existing Formalized Mathematics articles link to migrated modules - by updating old links now, when articles are revised, or not at all?

9.4 - What Is Preserved, What We Ask

Preserved:

- Formalized Mathematics remains a real journal with review and written explanations;
- proposed origin metadata links migrated items to MML citations; the migration mapping still needs to be defined.

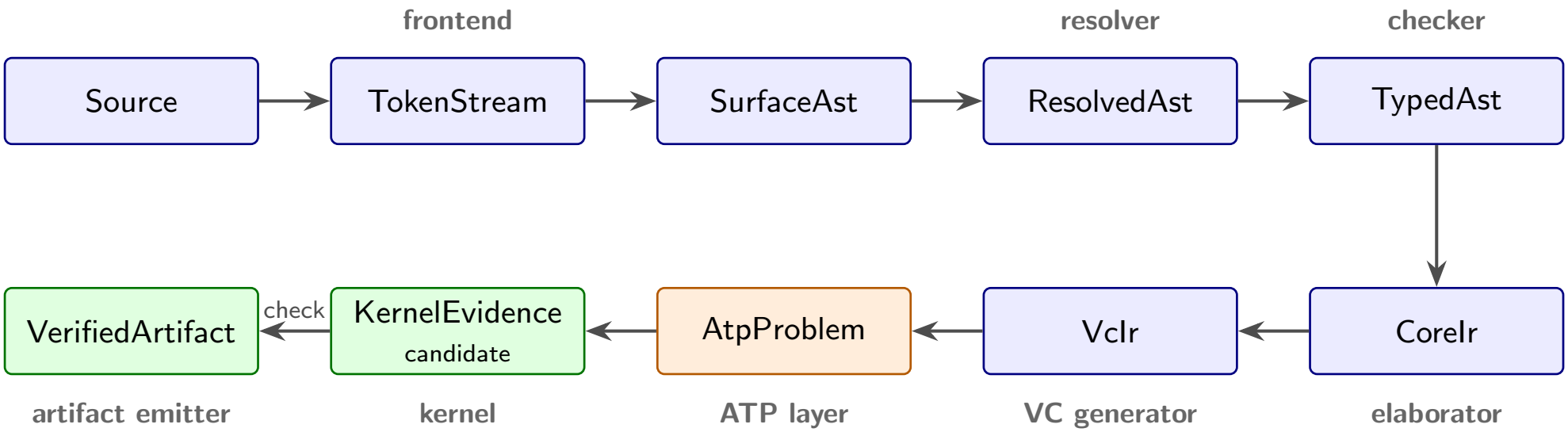
Questions for Bialystok:

- Which identity should be primary in user-facing citations: article label, library FQN, or origin id?
- How should existing Formalized Mathematics articles link to migrated modules - by updating old links now, when articles are revised, or not at all?

Presenter script: Read aloud:

- Formalized Mathematics remains a real journal with review and written explanations;
- proposed origin metadata links migrated items to MML citations; the migration mapping still needs to be defined.
- Which identity should be primary in user-facing citations: article label, library FQN, or origin id?
- How should existing Formalized Mathematics articles link to migrated modules - by updating old links now, when articles are revised, or not at all?

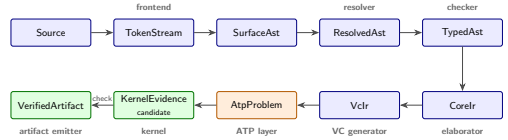
10.1 - The Core ATP Path



Message:

- every boundary states who owns a fact, which artifact records it, and what must be recomputed when it changes. That is the main purpose.

10.1 - The Core ATP Path



Message:

- every boundary states who owns a fact, which artifact records it, and what must be recomputed when it changes. That is the main purpose.

Presenter script: Say:

- The diagram shows the ATP path. Only obligations still open after deterministic discharge go to ATP; earlier discharge also needs evidence.

Read aloud:

- every boundary states who owns a fact, which artifact records it, and what must be recomputed when it changes. That is the main purpose.

10.2 - Responsibility Split

deep dive

Layer	Responsibility
frontend	lexing, parsing, recovery
resolver	imports, names, labels, namespaces
checker	soft types, clusters, registrations, overloads
elaborator	core logical representation
VC generator	proof and algorithm obligations
ATP layer plus kernel	untrusted search, then acceptance by checking
artifact emitter	stable outputs for tools and dependents

deep dive

10.2 - Responsibility Split
deep dive

Layer	Responsibility
frontend	lexing, parsing, recovery
resolver	imports, names, labels, namespaces
checker	soft types, clusters, registrations, overloads
elaborator	core logical representation
VC generator	proof and algorithm obligations
ATP layer plus kernel	untrusted search, then acceptance by checking
artifact emitter	stable outputs for tools and dependents

Presenter script: For the table, read the row labels first, then the design reason. Introduce the next topic: Responsibility Split. Point to the example, question, or diagram before you continue.

10.3 - Pipeline Stages For The Eight Stories

Story	Pipeline stage
dependencies	resolver, package manager
structures and automation	checker (inheritance and cluster graphs, traces)
search vs trust	ATP layer, kernel, formula/substitution evidence
scale	artifacts, fingerprints, scheduler
templates	elaborator (checked instantiation)
algorithms	VC generator, MVM
publication	artifact emitter, doc generation

Message:

- The eight stories describe one pipeline. Each starts with a different problem that users face.

10.3 - Pipeline Stages For The Eight Stories

Story	Pipeline stage
dependencies	resolver, package manager
structures and automation	checker (inheritance and cluster graphs, traces)
search vs trust	ATP layer, kernel, formula/substitution evidence
scale	artifacts, fingerprints, scheduler
templates	elaborator (checked instantiation)
algorithms	VC generator, MVM
publication	artifact emitter, doc generation

Message:

- The eight stories describe one pipeline. Each starts with a different problem that users face.

Presenter script: Read aloud:

- The eight stories describe one pipeline. Each starts with a different problem that users face.

For the table, read the row labels first, then the design reason.

10.4 - Testing The Trust Boundary

deep dive

Key Phrase:

Reject what must not pass
before
Accept everything that should pass

Key Points:

- soundness bugs have higher priority than parser gaps. Tests near the kernel first focus on malformed and failing evidence;
- with these tests in place, we add coverage for accepted language forms.

deep dive

10.4 - Testing The Trust Boundary
deep dive

Key Phrase:

Reject what must not pass
before
Accept everything that should pass

Key Points:

- soundness bugs have higher priority than parser gaps. Tests near the kernel first focus on malformed and failing evidence;
- with these tests in place, we add coverage for accepted language forms.

Presenter script: Read aloud:

- soundness bugs have higher priority than parser gaps. Tests near the kernel first focus on malformed and failing evidence;
- with these tests in place, we add coverage for accepted language forms.

After the example, explain which design obligation the syntax shows.

11.1 - Where The Project Stands Today

Key Points:

- bilingual language specification: 24 chapters plus appendices, English canonical with Japanese companions (`doc/spec/`);
- 24 draft architecture documents covering the pipeline, kernel, kernel evidence, and AI agent interface (`doc/design/architecture/`);
- a Rust workspace of 20 crates - lexer, parser, resolver, checker, VC generator, ATP bridge, kernel, build system - roughly 400k lines including tests;
- focused audits completed in 2026: kernel soundness, template logic encoding, SAT solver dependency;
- the roadmap is split into small tasks that can be verified separately.

Message:

- The planned end-of-2026 alpha is a milestone in work already under way.

11.1 - Where The Project Stands Today

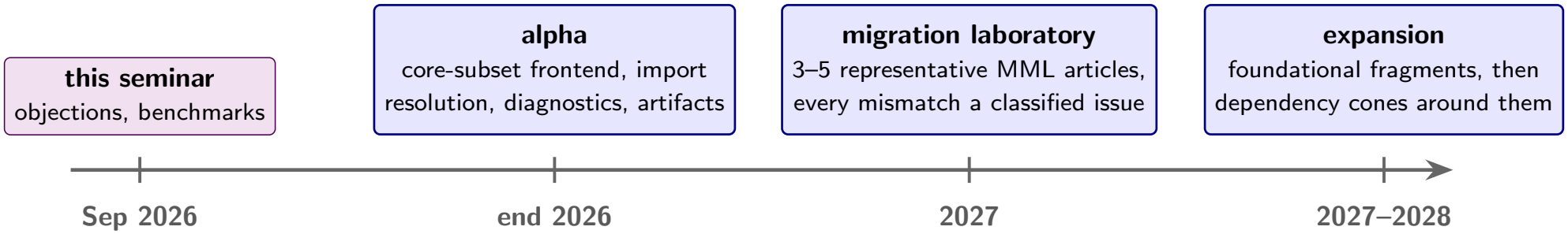
Key Points:
- bilingual language specification: 24 chapters plus appendices, English canonical with Japanese companions (doc/spec/);
- 24 draft architecture documents covering the pipeline, kernel, kernel evidence, and AI agent interface (doc/design/architecture/);
- a Rust workspace of 20 crates - lexer, parser, resolver, checker, VC generator, ATP bridge, kernel, build system - roughly 400k lines including tests;
- focused audits completed in 2026: kernel soundness, template logic encoding, SAT solver dependency;
- the roadmap is split into small tasks that can be verified separately.

Message:
- The planned end-of-2026 alpha is a milestone in work already under way.

Presenter script: Read aloud:

- bilingual language specification: 24 chapters plus appendices, English canonical with Japanese companions (doc/spec/);
- 24 draft architecture documents covering the pipeline, kernel, kernel evidence, and AI agent interface (doc/design/architecture/);
- a Rust workspace of 20 crates - lexer, parser, resolver, checker, VC generator, ATP bridge, kernel, build system - roughly 400k lines including tests;
- focused audits completed in 2026: kernel soundness, template logic encoding, SAT solver dependency;
- the roadmap is split into small tasks that can be verified separately.

11.2 - Migration Needs Research

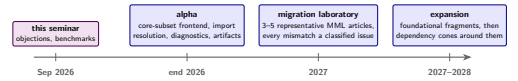


1. End of 2026, alpha: core-subset frontend and parser, import and module resolution prototype, structured diagnostics, early artifacts.
2. 2027, migration laboratory: translate 3-5 representative MML articles by hand and by script. Classify and record every mismatch.
3. 2027-2028, expansion: foundational set and relation fragments; then algebraic structures and dependency cones around successful fragments.

Non-goals for the alpha:

- full MML verification, final compatibility layer, stable AI protocol.

11.2 - Migration Needs Research



- End of 2026, alpha: core-subset frontend and parser, import and module resolution prototype, structured diagnostics, early artifacts.
- 2027, migration laboratory: translate 3-5 representative MML articles by hand and by script. Classify and record every mismatch.
- 2027-2028, expansion: foundational set and relation fragments; then algebraic structures and dependency cones around successful fragments.

Non-goals for the alpha:

- full MML verification, final compatibility layer, stable AI protocol.

Presenter script: Read aloud:

- End of 2026, alpha: core-subset frontend and parser, import and module resolution prototype, structured diagnostics, early artifacts.
- 2027, migration laboratory: translate 3-5 representative MML articles by hand and by script. Classify and record every mismatch.
- 2027-2028, expansion: foundational set and relation fragments; then algebraic structures and dependency cones around successful fragments.
- full MML verification, final compatibility layer, stable AI protocol.

11.3 - What We Will Measure

deep dive

Key Points:

- translated articles and lines; accepted parser subset;
- resolved imports versus unresolved dependencies;
- obligations closed deterministically versus by ATP with kernel evidence;
- memory and wall-clock per module, incremental versus clean;
- compatibility decisions that required human judgment.

deep dive

11.3 - What We Will Measure
deep dive

Key Points:

- translated articles and lines; accepted parser subset;
- resolved imports versus unresolved dependencies;
- obligations closed deterministically versus by ATP with kernel evidence;
- memory and wall-clock per module, incremental versus clean;
- compatibility decisions that required human judgment.

Presenter script: Read aloud:

- translated articles and lines; accepted parser subset;
- resolved imports versus unresolved dependencies;
- obligations closed deterministically versus by ATP with kernel evidence;
- memory and wall-clock per module, incremental versus clean;
- compatibility decisions that required human judgment.

11.4 - Compatibility Policy And Risks

deep dive

Policy:

- define origin mappings to preserve MML theorem identity;
- keep compatibility aliases where they help migration;
- record every difference from old behavior with a reason and a test.

Risk	How to reduce it
compatibility work takes all our time	small representative parts first; no all-at-once translation
registrations behave differently	trace artifacts and comparison reports early
package layout breaks journal links	origin metadata, article-to-library identifiers
AI edits hide migration mistakes	Red edits stay forbidden to ordinary agents; verifier artifacts required

deep dive

Presenter script: Read aloud:

- define origin mappings to preserve MML theorem identity;
- keep compatibility aliases where they help migration;
- record every difference from old behavior with a reason and a test.

For the table, read the row labels first, then the design reason.

11.4 - Compatibility Policy And Risks
deep dive

Policy:

- define origin mappings to preserve MML theorem identity;
- keep compatibility aliases where they help migration;
- record every difference from old behavior with a reason and a test.

Risk	How to reduce it
compatibility work takes all our time	small representative parts first; no all-at-once translation
registrations behave differently	trace artifacts and comparison reports early
package layout breaks journal links	origin metadata, article-to-library identifiers
AI edits hide migration mistakes	Red edits stay forbidden to ordinary agents; verifier artifacts required

11.5 - What September 2026 Should Produce

Key Points:

- a list of migration benchmark articles in priority order;
- agreement on compatibility metadata needs;
- review notes on the eight stories, especially structures and clusters;
- a first paper outline;
- a shared list of objections and risks.

11.5 - What September 2026 Should Produce

Key Points:
- a list of migration benchmark articles in priority order;
- agreement on compatibility metadata needs;
- review notes on the eight stories, especially structures and clusters;
- a first paper outline;
- a shared list of objections and risks.

Presenter script: Read aloud:

- a list of migration benchmark articles in priority order;
- agreement on compatibility metadata needs;
- review notes on the eight stories, especially structures and clusters;
- a first paper outline;
- a shared list of objections and risks.

11.6 - What We Ask Of You

Questions:

- Which MML articles are small but structurally representative?
- Which idioms matter to the Mizar community, beyond their technical use?
- Which of the eight stories is most wrong, and why?
- What migration result would convince the community that Evo is a serious project?

11.6 - What We Ask Of You

Questions:
- Which MML articles are small but structurally representative?
- Which idioms matter to the Mizar community, beyond their technical use?
- Which of the eight stories is most wrong, and why?
- What migration result would convince the community that Evo is a serious project?

Presenter script: Read aloud:

- Which MML articles are small but structurally representative?
- Which idioms matter to the Mizar community, beyond their technical use?
- Which of the eight stories is most wrong, and why?
- What migration result would convince the community that Evo is a serious project?

12.1 - The Main Question, Again

Slide question:

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

Slide question:

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

Presenter script: Say:

- Return to the opening example: the structure that still reads as Mizar.
- Ask for objections to each story, as well as to the overall plan.

12.2 - Closing

Final slide:

Update Mizar Evo where a larger library needs it.
Keep the current design where it defines Mizar's mathematical identity.

Final slide:

Update Mizar Evo where a larger library needs it.
Keep the current design where it defines Mizar's mathematical identity.

Presenter script: After the example, explain which design obligation the syntax shows. Introduce the next topic: Closing. Point to the example, question, or diagram before you continue.

Backup A. Prepared Exact Examples (1/2)

Prepared short excerpts for Beamer conversion:

Purpose	Source	Lines	Used in
Structure definition	`algstr_0.miz`	37-40	Frames 0.2, 3.1
Article environment	`algstr_0.miz`	15-25	Frame 2.1
Registration/cluster	`algstr_0.miz`	104-109	Frame 4.1

Prepared short excerpts for Beamer conversion:

Purpose	Source	Lines	Used in
Proof citation	`struct_0.miz`	637-643	Frame 5.4
Induction scheme	`nat_1.miz`	near 90	Frame 7.1

Source URLs:

- `ALGSTR_0`: <https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz>
- `STRUCT_0`: <https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz>
- `NAT_1`: <https://mizar.uwb.edu.pl/version/current/mml/nat_1.miz>

Backup A. Prepared Exact Examples (1/2)

Prepared short excerpts for Beamer conversion:

Purpose	Source	Lines	Used in
Structure definition	algstr_0.miz	37-40	Frames 0.2, 3.1
Article environment	algstr_0.miz	15-25	Frame 2.1
Registration/cluster	algstr_0.miz	104-109	Frame 4.1

Prepared short excerpts for Beamer conversion:

Purpose	Source	Lines	Used in
Proof citation	struct_0.miz	637-643	Frame 5.4
Induction scheme	nat_1.miz	near 90	Frame 7.1

Source URLs:

- ALGSTR_0: <https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz>
- STRUCT_0: <https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz>
- NAT_1: <https://mizar.uwb.edu.pl/version/current/mml/nat_1.miz>

Presenter script: Read aloud:

- ALGSTR_0: <https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz>
- STRUCT_0: <https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz>
- NAT_1: <https://mizar.uwb.edu.pl/version/current/mml/nat_1.miz>

For the table, read the row labels first, then the design reason.

Backup A. Prepared Exact Examples (2/2)

Attribution note:

- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.

Attribution note:
- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.

Presenter script: Read aloud:

- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.

Backup B. Specification Reference Map (1/2)

The slides show examples only. The specification is the authority for grammar and semantics. This map replaces the EBNF shown in earlier drafts.

Topic	**Specification source (under `doc/spec/en/`)**
Modules and imports	`12.modules_and_namespaces.md`
Structures and inheritance	`05.structures.md`
Attributes and modes	`06.attributes.md`, `07.modes.md`
Registrations and reductions	`17.clusters_and_registrations.md`
Templates and schemes	`18.templates.md`
Algorithms and MVM	`20.algorithm_and_verification.md`
ATP and kernel evidence	`21.source_code_annotation_and_atp.md`

Backup B. Specification Reference Map (1/2)

The slides show examples only. The specification is the authority for grammar and semantics. This map replaces the EBNF shown in earlier drafts.

Topic	Specification source (under doc/spec/en/)
Modules and imports	12.modules_and_namespaces.md
Structures and inheritance	05.structures.md
Attributes and modes	06.attributes.md, 07.modes.md
Registrations and reductions	17.clusters_and_registrations.md
Templates and schemes	18.templates.md
Algorithms and MVM	20.algorithm_and_verification.md
ATP and kernel evidence	21.source_code_annotation_and_atp.md

Presenter script: Read aloud:

- The slides show examples only.
- The specification is the authority for grammar and semantics.
- This map replaces the EBNF shown in earlier drafts.

For the table, read the row labels first, then the design reason.

Backup B. Specification Reference Map (2/2)

Topic	**Specification source (under `doc/spec/en/`)**
Packages and artifacts	`23.package_management_and_build_system.md`
Cross-chapter grammar	`appendix_a.grammar_summary.md`
Worked library sketches	`sample_codes.md`

Backup B. Specification Reference Map (2/2)

Topic	Specification source (under doc/spec/en/)
Packages and artifacts	23_package_management_and_build_system.md
Cross-chapter grammar	appendix_a_grammar_summary.md
Worked library sketches	sample_codes.md

Presenter script: For the table, read the row labels first, then the design reason. Introduce the next topic: Backup B. Specification Reference Map (2/2). Point to the example, question, or diagram before you continue.

Backup C. Diagram List (1/2)

Required diagrams for the final deck (`figures/*.tex`, TikZ standalone;

build each with `pdflatex` inside `figures/`):

1. Three pressures on a proven design (Part 1). [done: `figures/three_pressures.pdf`, used in frame 1.5]
2. Environment-to-import migration (story 1). [done: `figures/environ_migration.pdf`, used in frame 2.5]
3. Structure inheritance and diamond coherence (story 2). [done: `figures/diamond_inheritance.pdf`, used in frame 3.5]
4. Reasoning boundary: semantics / ATP search / kernel checking (story 4). [done: `figures/reasoning_boundary.pdf`, used in frame 5.2]
5. KernelEvidence and the trusted SAT check (story 4). [done: `figures/certificate_replay.pdf`, used in frame 5.3]

Backup C. Diagram List (1/2)

Required diagrams for the final deck (figures/*.tex, TikZ standalone;
build each with pdflatex inside figures/):

- Three pressures on a proven design (Part 1). [done: figures/three_pressures.pdf, used in frame 1.5]
- Environment-to-import migration (story 1). [done: figures/environ_migration.pdf, used in frame 2.5]
- Structure inheritance and diamond coherence (story 2). [done: figures/diamond_inheritance.pdf, used in frame 3.5]
- Reasoning boundary: semantics / ATP search / kernel checking (story 4). [done: figures/reasoning_boundary.pdf, used in frame 5.2]
- KernelEvidence and the trusted SAT check (story 4). [done: figures/certificate_replay.pdf, used in frame 5.3]

Presenter script: Read aloud:

- Required diagrams for the final deck (figures/*.tex, TikZ standalone;
- Three pressures on a proven design (Part 1). [done: figures/three_pressures.pdf, used in frame 1.5]
- Environment-to-import migration (story 1). [done: figures/environ_migration.pdf, used in frame 2.5]
- Structure inheritance and diamond coherence (story 2). [done: figures/diamond_inheritance.pdf, used in frame 3.5]
- Reasoning boundary: semantics / ATP search / kernel checking (story 4). [done: figures/reasoning_boundary.pdf, used in frame 5.2]

Backup C. Diagram List (2/2)

build each with `pdflatex` inside `figures/`):

1. Incremental fingerprint graph (story 5). [done: `figures/fingerprint_graph.pdf`, used in frame 6.2]
2. Formalized Mathematics article-to-library link model (story 8). [done: `figures/fm_links.pdf`, used in frame 9.2]
3. Core ATP path with responsibility groups (Part 10). [done: `figures/pipeline.pdf`, used in frame 10.1]
4. Roadmap timeline (Part 11). [done: `figures/roadmap_timeline.pdf`, used in frame 11.1]

Backup C. Diagram List (2/2)

build each with pdflatex inside figures/):
- Incremental fingerprint graph (story 5). [done: figures/fingerprint_graph.pdf, used in frame 6.2]
- Formalized Mathematics article-to-library link model (story 8). [done: figures/fm_links.pdf, used in frame 9.2]
- Core ATP path with responsibility groups (Part 10). [done: figures/pipeline.pdf, used in frame 10.1]
- Roadmap timeline (Part 11). [done: figures/roadmap_timeline.pdf, used in frame 11.1]

Presenter script: Read aloud:

- Incremental fingerprint graph (story 5). [done: figures/fingerprint_graph.pdf, used in frame 6.2]
- Formalized Mathematics article-to-library link model (story 8). [done: figures/fm_links.pdf, used in frame 9.2]
- Core ATP path with responsibility groups (Part 10). [done: figures/pipeline.pdf, used in frame 10.1]
- Roadmap timeline (Part 11). [done: figures/roadmap_timeline.pdf, used in frame 11.1]

Backup D. Possible Paper Outline (1/2)

Possible paper title:

Mizar Evo: Readable, AI-Ready, and Scalable Formal Mathematics

Possible sections:

1. Introduction: why Mizar needs evolution now.
2. Mizar as baseline: readability, MML, Formalized Mathematics.
3. Design principles and the three goals.
4. Language evolution: dependencies, structures, registrations, templates.
5. Verifier architecture, kernel evidence, and the trusted SAT checker.
6. Verified computation and the MVM.
7. AI-safe proof development.

Backup D. Possible Paper Outline (1/2)

Possible paper title:

Mizar Evo: Readable, AI-Ready, and Scalable Formal Mathematics

Possible sections:

- Introduction: why Mizar needs evolution now.
- Mizar as baseline: readability, MML, Formalized Mathematics.
- Design principles and the three goals.
- Language evolution: dependencies, structures, registrations, templates.
- Verifier architecture, kernel evidence, and the trusted SAT checker.
- Verified computation and the MVM.
- AI-safe proof development.

Presenter script: Read aloud:

- Introduction: why Mizar needs evolution now.
- Mizar as baseline: readability, MML, Formalized Mathematics.
- Design principles and the three goals.

After the example, explain which design obligation the syntax shows.

Backup D. Possible Paper Outline (2/2)

Possible sections:

1. Package-based library and publication workflow.
2. Migration plan and evaluation measures.
3. Related work and plans for working together.

Possible sections:

- Package-based library and publication workflow.
- Migration plan and evaluation measures.
- Related work and plans for working together.

Presenter script: Read aloud:

- Package-based library and publication workflow.
- Migration plan and evaluation measures.
- Related work and plans for working together.

Backup E. Reviewer Checklist

Use this checklist before converting to Beamer:

- Does every story open with a real cost, not a feature announcement?
- Does every criticism explain why the current practice was useful?
- Does every code example carry its status label (exact MML excerpt, specification example, or sketch)?
- Is every exact excerpt attributed with article and line numbers?
- Does every story end with questions the audience can actually answer?
- Are Red AI edits clearly forbidden?
- Are migration claims measurable?

Use this checklist before converting to Beamer:

- Is EBNF absent from all frames?

Use this checklist before converting to Beamer:
- Does every story open with a real cost, not a feature announcement?
- Does every criticism explain why the current practice was useful?
- Does every code example carry its status label (exact MML excerpt, specification example, or sketch)?
- Is every exact excerpt attributed with article and line numbers?
- Does every story end with questions the audience can actually answer?
- Are Red AI edits clearly forbidden?
- Are migration claims measurable?

Use this checklist before converting to Beamer:
- Is EBNF absent from all frames?

Presenter script: Read aloud:

- Does every story open with a real cost, not a feature announcement?
- Does every criticism explain why the current practice was useful?
- Does every code example carry its status label (exact MML excerpt, specification example, or sketch)?
- Is every exact excerpt attributed with article and line numbers?
- Does every story end with questions the audience can actually answer?

Backup F. Possible Objections (1/2)

Prepared answers to other possible objections:

Objection	Prepared answer
Why not improve current Mizar incrementally?	The problems concern boundaries: article granularity, monolithic trust, and implicit environments. These boundaries cannot change in small steps. The rest of the design stays close to Mizar.
What happens to authorship and credit of MML articles?	We propose origin mappings that preserve identity and credit during migration. The mapping format is still open. The `pub` namespace keeps published articles unchanged.
Is this a fork of the community?	It is a proposal to the community; this visit is its first review. The namespace governance model assumes that the Mizar team controls the `mml` root.

Backup F. Possible Objections (1/2)

Prepared answers to other possible objections:

Objection	Prepared answer
Why not improve current Mizar incrementally?	The problems concern boundaries: article granularity, monolithic trust, and implicit environments. These boundaries cannot change in small steps. The rest of the design stays close to Mizar.
What happens to authorship and credit of MML articles?	We propose origin mappings that preserve identity and credit during migration. The mapping format is still open. The pub namespace keeps published articles unchanged.
Is this a fork of the community?	It is a proposal to the community; this visit is its first review. The namespace governance model assumes that the Mizar team controls the mml root.

Presenter script: Say:

- Use these answers only if someone asks these questions.

For the table, read the row labels first, then the design reason.

Backup F. Possible Objections (2/2)

Prepared answers to other possible objections:

Objection	Prepared answer
What about GPL / CC-BY-SA obligations?	Migration preserves license and attribution of MML content; toolchain licensing is open for discussion.
Are the claims about AI too strong?	AI assistance is an optional layer that never enters the trusted base; every proposal also works without it.
Why a new kernel instead of the existing checker?	Formula/substitution evidence needs a small trusted core, including a SAT checker. Mizar Evo's specification defines the obligations; legacy behavior guides migration comparisons.

Backup F. Possible Objections (2/2)

Prepared answers to other possible objections:

Objection	Prepared answer
What about GPL / CC-BY-SA obligations?	Migration preserves license and attribution of MML content; toolchain licensing is open for discussion.
Are the claims about AI too strong?	AI assistance is an optional layer that never enters the trusted base; every proposal also works without it.
Why a new kernel instead of the existing checker?	Formula/substitution evidence needs a small trusted core, including a SAT checker. Mizar Evo's specification defines the obligations; legacy behavior guides migration comparisons.

Presenter script: For the table, read the row labels first, then the design reason. Introduce the next topic: Backup F. Possible Objections (2/2). Point to the example, question, or diagram before you continue.